Dokumentasi Resmi & Sintaks TypeScript (5.x Up-to-Date)

Bahasa: TypeScript | Total Chunk/Dokumen: 216 | Disiapkan untuk RAG System

[typescript_001] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): Overview

Sumber: ht

Each and every value in JavaScript has a set of behaviors you can observe from running different operations.
That sounds abstract, but as a quick example, consider some operations we might run on a variable named `message`.
If we break this down, the first runnable line of code accesses a property called `toLowerCase` and then calls it.
The second one tries to call `message` directly.
But assuming we don't know the value of `message` - and that's pretty common - we can't reliably say what results we'll get from trying to run any of this code.
The behavior of each operation depends entirely on what value we had in the first place.
- Is `message` callable?
- Does it have a property called `toLowerCase` on it?
- If it does, is `toLowerCase` even callable?
- If both of these values are callable, what do they return?

The answers to these questions are usually things we keep in our heads when we write JavaScript, and we have to hope we got all the details right.
Let's say `message` was defined in the following way.
As you can probably guess, if we try to run `message.toLowerCase()`, we'll get the same string only in lower-case.
What about that second line of code?
If you're familiar with JavaScript, you'll know this fails with an exception:
It'd be great if we could avoid mistakes like this.
When we run our code, the way that our JavaScript runtime chooses what to do is by figuring out the _type_ of the value - what sorts of behaviors and capabilities it has.
That's part of what that `TypeError` is alluding to - it's saying that the string `"Hello World!"` cannot be called as a function.
For some values, such as the primitives `string` and `number`, we can identify their type at runtime using the `typeof` operator.
But for other things like functions, there's no corresponding

Contoh Kode / Perintah CLI:

```
// Accessing the property 'toLowerCase'
// on 'message' and then calling it
message.toLowerCase();

// Calling 'message'
message();

// ----------------------------------------

const message = "Hello World!";

// ----------------------------------------

It'd be great if we could avoid mistakes like this.

When we run our code, the way that our JavaScript runtime chooses what to do is by figuring out the _type_ of the value - what sorts of behaviors and capabilities it has.
That's part of what that `TypeError` is alluding to - it's saying that the string `"Hello World!"` cannot be called as a function.
```

```
For some values, such as the primitives `string` and `number`, we can identify their type at runtime using the `typeof` operator.
But for other things like functions, there's no corresponding runtime mechanism to identify their types.
For example, consider this function:
```

[typescript_002] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): Static type-checking

Sumber: ht

Static type-checking
Think back to that `TypeError` we got earlier from trying to call a `string` as a function.
Most people don't like to get any sorts of errors when running their code - those are considered bugs!
And when we write new code, we try our best to avoid introducing new bugs.
If we add just a bit of code, save our file, re-run the code, and immediately see the error, we might be able to isolate the problem quickly; but that's not always the case.
We might not have tested the feature thoroughly enough, so we might never actually run into a potential error that would be thrown!
Or if we were lucky enough to witness the error, we might have ended up doing large refactorings and adding a lot of different code that we're forced to dig through.
Ideally, we could have a tool that helps us find these bugs _before_ our code runs.
That's what a static type-checker like TypeScript does.
Static type systems describe the shapes and behaviors of what our values will be when we run our programs.
A type-checker like TypeScript uses that information and tells us when things might be going off the rails.
Running that last sample with TypeScript will give us an error message before we run the code in the first place.

[typescript_003] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): Non-exception Failures

Sumber: ht

Non-exception Failures
So far we've been discussing certain things like runtime errors - cases where the JavaScript runtime tells us that it thinks something is nonsensical.
Those cases come up because the ECMAScript specification has explicit instructions on how the language should behave when it runs into something unexpected.
For example, the specification says that trying to call something that isn't callable should throw an error.
Maybe that sounds like "obvious behavior", but you could imagine that accessing a property that doesn't exist on an object should throw an error too.
Instead, JavaScript gives us different behavior and returns the value `undefined`:
Ultimately, a static type system has to make the call over what code should be flagged as an error in its system, even if it's "valid" JavaScript that won't immediately throw an error.
In TypeScript, the following code produces an error about `location` not being defined:
While sometimes that implies a trade-off in what you can express, the intent is to catch legitimate bugs in our programs.
And TypeScript catches _a lot_ of legitimate bugs.
For example: typos,
uncalled functions,
or basic logic errors.

Contoh Kode / Perintah CLI:

```
const user = {
  name: "Daniel",
  age: 26,
};

user.location; // returns undefined
```

```
// ----------------------------------------

While sometimes that implies a trade-off in what you can express, the intent is to catch legitimate bugs
in our programs.
And TypeScript catches _a lot_ of legitimate bugs.

For example: typos,

// ----------------------------------------

uncalled functions,
```

[typescript_004] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): Types for Tooling

Sumber: ht

Types for Tooling
TypeScript can catch bugs when we make mistakes in our code.
That's great, but TypeScript can _also_ prevent us from making those mistakes in the first place.
The type-checker has information to check things like whether we're accessing the right properties on variables and other properties.
Once it has that information, it can also start _suggesting_ which properties you might want to use.
That means TypeScript can be leveraged for editing code too, and the core type-checker can provide error messages and code completion as you type in the editor.
That's part of what people often refer to when they talk about tooling in TypeScript.
TypeScript takes tooling seriously, and that goes beyond completions and errors as you type.
An editor that supports TypeScript can deliver "quick fixes" to automatically fix errors, refactorings to easily re-organize code, and useful navigation features for jumping to definitions of a variable, or finding all references to a given variable.
All of this is built on top of the type-checker and is fully cross-platform, so it's likely that your favorite editor has TypeScript support available.

[typescript_005] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): `tsc`, the TypeScript compiler

Sumber: ht

`tsc`, the TypeScript compiler
We've been talking about type-checking, but we haven't yet used our type-_checker_.
Let's get acquainted with our new friend `tsc`, the TypeScript compiler.
First we'll need to grab it via npm.
> This installs the TypeScript Compiler `tsc` globally.
> You can use `npx` or similar tools if you'd prefer to run `tsc` from a local `node_modules` package instead.
Now let's move to an empty folder and try writing our first TypeScript program: `hello.ts`:
Notice there are no frills here; this "hello world" program looks identical to what you'd write for a "hello world" program in JavaScript.
And now let's type-check it by running the command `tsc` which was installed for us by the `typescript` package.
Tada!
Wait, "tada" _what_ exactly?
We ran `tsc` and nothing happened!
Well, there were no type errors, so we didn't get any output in our console since there was nothing to report.
But check again - we got some _file_ output instead.
If we look in our current directory, we'll see a `hello.js` file next to `hello.ts`.
That's the output from our `hello.ts` file after `tsc` _compiles_ or _transforms_ it into a plain JavaScript file.
And if we check the contents, we'll see what TypeScript spits out after it processes a `.ts` file:

In this case, there was very little for TypeScript to transform, so it looks identical to what we wrote.
The compiler tries to emit clean readable code that looks like something a person would write.
While that's not always so easy, TypeScript indents consistently, is mindful of when our code spans across different lines of code, and tries to keep comments around.
What about if we _did_ introduce a type-checking error?
Let's rewrite `hello.ts`:
If we run `tsc hello.ts` again, notice that we get an error on the command line

Contoh Kode / Perintah CLI:

```
npm install -g typescript

// ----------------------------------------

Notice there are no frills here; this "hello world" program looks identical to what you'd write for a "hello world" program in JavaScript.
And now let's type-check it by running the command `tsc` which was installed for us by the `typescript` package.

// ----------------------------------------

Tada!

Wait, "tada" _what_ exactly?
We ran `tsc` and nothing happened!
Well, there were no type errors, so we didn't get any output in our console since there was nothing to report.

But check again - we got some _file_ output instead.
If we look in our current directory, we'll see a `hello.js` file next to `hello.ts`.
That's the output from our `hello.ts` file after `tsc` _compiles_ or _transforms_ it into a plain JavaScript file.
And if we check the contents, we'll see what TypeScript spits out after it processes a `.ts` file:
```

[typescript_006] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): Emitting with Errors

Sumber: ht

Emitting with Errors
One thing you might not have noticed from the last example was that our `hello.js` file changed again.
If we open that file up then we'll see that the contents still basically look the same as our input file.
That might be a bit surprising given the fact that `tsc` reported an error about our code, but this is based on one of TypeScript's core values: much of the time, _you_ will know better than TypeScript.
To reiterate from earlier, type-checking code limits the sorts of programs you can run, and so there's a tradeoff on what sorts of things a type-checker finds acceptable.
Most of the time that's okay, but there are scenarios where those checks get in the way.
For example, imagine yourself migrating JavaScript code over to TypeScript and introducing type-checking errors.
Eventually you'll get around to cleaning things up for the type-checker, but that original JavaScript code was already working!
Why should converting it over to TypeScript stop you from running it?
So TypeScript doesn't get in your way.
Of course, over time, you may want to be a bit more defensive against mistakes, and make TypeScript act a bit more strictly.
In that case, you can use the `noEmitOnError` compiler option.
Try changing your `hello.ts` file and running `tsc` with that flag:
You'll notice that `hello.js` never gets updated.

Contoh Kode / Perintah CLI:

```
tsc --noEmitOnError hello.ts
```

[typescript_007] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): Explicit Types

Sumber: ht

Explicit Types
Up until now, we haven't told TypeScript what `person` or `date` are.
Let's edit the code to tell TypeScript that `person` is a `string`, and that `date` should be a `Date` object.
We'll also use the `toDateString()` method on `date`.
What we did was add _type annotations_ on `person` and `date` to describe what types of values `greet` can be called with.
You can read that signature as "`greet` takes a `person` of type `string`, and a `date` of type `Date`".
With this, TypeScript can tell us about other cases where `greet` might have been called incorrectly.
For example...
Huh?
TypeScript reported an error on our second argument, but why?
Perhaps surprisingly, calling `Date()` in JavaScript returns a `string`.
On the other hand, constructing a `Date` with `new Date()` actually gives us what we were expecting.
Anyway, we can quickly fix up the error:
Keep in mind, we don't always have to write explicit type annotations.
In many cases, TypeScript can even just _infer_ (or "figure out") the types for us even if we omit them.
Even though we didn't tell TypeScript that `msg` had the type `string` it was able to figure that out.
That's a feature, and it's best not to add annotations when the type system would end up inferring the same type anyway.
> Note: The message bubble inside the previous code sample is what your editor would show if you had hovered over the word.

Contoh Kode / Perintah CLI:

```
What we did was add _type annotations_ on `person` and `date` to describe what types of values `greet` can be called with.
You can read that signature as "`greet` takes a `person` of type `string`, and a `date` of type `Date`".

With this, TypeScript can tell us about other cases where `greet` might have been called incorrectly.
For example...

// ----------------------------------------

Huh?
TypeScript reported an error on our second argument, but why?

Perhaps surprisingly, calling `Date()` in JavaScript returns a `string`.
On the other hand, constructing a `Date` with `new Date()` actually gives us what we were expecting.

Anyway, we can quickly fix up the error:

// ----------------------------------------

Keep in mind, we don't always have to write explicit type annotations.
In many cases, TypeScript can even just _infer_ (or "figure out") the types for us even if we omit them.
```

[typescript_008] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): Erased Types

Sumber: ht

Erased Types
Let's take a look at what happens when we compile the above function `greet` with `tsc` to output JavaScript:
Notice two things here:
1. Our `person` and `date` parameters no longer have type annotations.
2. Our "template string" - that string that used backticks (the `` ` `` character) - was converted to plain strings with concatenations.
More on that second point later, but let's now focus on that first point.
Type annotations aren't part of JavaScript (or ECMAScript to be pedantic), so there really aren't any browsers or other runtimes that can just run TypeScript unmodified.
That's why TypeScript needs a compiler in the first place - it needs some way to strip out or transform any TypeScript-specific code so that you can run it.
Most TypeScript-specific code gets erased away, and likewise, here our type annotations were completely erased.
> **Remember**: Type annotations never change the runtime behavior of your program.

[typescript_009] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): Downleveling

Sumber: ht

Downleveling
One other difference from the above was that our template string was rewritten from
to
Why did this happen?
Template strings are a feature from a version of ECMAScript called ECMAScript 2015 (a.k.a. ECMAScript 6, ES2015, ES6, etc. - _don't ask_).
TypeScript has the ability to rewrite code from newer versions of ECMAScript to older ones such as ECMAScript 3 or ECMAScript 5 (a.k.a. ES5).
This process of moving from a newer or "higher" version of ECMAScript down to an older or "lower" one is sometimes called _downleveling_.
By default TypeScript targets ES5, an extremely old version of ECMAScript.
We could have chosen something a little bit more recent by using the `target` option.
Running with `--target es2015` changes TypeScript to target ECMAScript 2015, meaning code should be able to run wherever ECMAScript 2015 is supported.
So running `tsc --target es2015 hello.ts` gives us the following output:
> While the default target is ES5, the great majority of current browsers support ES2015.
> Most developers can therefore safely specify ES2015 or above as a target, unless compatibility with certain ancient browsers is important.

Contoh Kode / Perintah CLI:

```
`Hello ${person}, today is ${date.toDateString()}!`;

// ----------------------------------------

"Hello ".concat(person, ", today is ").concat(date.toDateString(), "!");

// ----------------------------------------

function greet(person, date) {
  console.log(`Hello ${person}, today is ${date.toDateString()}!`);
}
greet("Maddison", new Date());
```

[typescript_010] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): Strictness

Sumber: ht

Strictness
Different users come to TypeScript looking for different things in a type-checker.
Some people are looking for a more loose opt-in experience which can help validate only some parts of their program, and still have decent tooling.
This is the default experience with TypeScript, where types are optional, inference takes the most lenient types, and there's no checking for potentially `null`/`undefined` values.
Much like how `tsc` emits in the face of errors, these defaults are put in place to stay out of your way.
If you're migrating existing JavaScript, that might be a desirable first step.
In contrast, a lot of users prefer to have TypeScript validate as much as it can straight away, and that's why the language provides strictness settings as well.
These strictness settings turn static type-checking from a switch (either your code is checked or not) into something closer to a dial.
The further you turn this dial up, the more TypeScript will check for you.
This can require a little extra work, but generally speaking it pays for itself in the long run, and enables more thorough checks and more accurate tooling.
When possible, a new codebase should always turn these strictness checks on.
TypeScript has several type-checking strictness flags that can be turned on or off, and all of our examples will be written with all of them enabled unless otherwise stated.
The `strict` flag in the CLI, or `"strict": true` in a `tsconfig.json` toggles them all on simultaneously, but we can opt out of them individually.
The two biggest ones you should know about are `noImplicitAny` and `strictNullChecks`.

[typescript_011] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): `noImplicitAny`

Sumber: ht

`noImplicitAny`
Recall that in some places, TypeScript doesn't try to infer types for us and instead falls back to the most lenient type: `any`.
This isn't the worst thing that can happen - after all, falling back to `any` is just the plain JavaScript experience anyway.
However, using `any` often defeats the purpose of using TypeScript in the first place.
The more typed your program is, the more validation and tooling you'll get, meaning you'll run into fewer bugs as you code.
Turning on the `noImplicitAny` flag will issue an error on any variables whose type is implicitly inferred as `any`.

[typescript_012] TypeScript - Basics & Tooling

The Basics (Type Checking, Types by Inference, Downleveling): `strictNullChecks`

Sumber: ht

`strictNullChecks`
By default, values like `null` and `undefined` are assignable to any other type.
This can make writing some code easier, but forgetting to handle `null` and `undefined` is the cause of countless bugs in the world - some consider it a billion dollar mistake!
The `strictNullChecks` flag makes handling `null` and `undefined` more explicit, and _spares_ us from worrying about whether we _forgot_ to handle `null` and `undefined`.

[typescript_013] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Overview

Sumber: ht

In this chapter, we'll cover some of the most common types of values you'll find in JavaScript code, and explain the corresponding ways to describe those types in TypeScript.
This isn't an exhaustive list, and future chapters will describe more ways to name and use other types.
Types can also appear in many more _places_ than just type annotations.
As we learn about the types themselves, we'll also learn about the places where we can refer to these types to form new constructs.
We'll start by reviewing the most basic and common types you might encounter when writing JavaScript or TypeScript code.
These will later form the core building blocks of more complex types.

[typescript_014] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): The primitives: `string`, `number`, and `boolean`

Sumber: ht

The primitives: `string`, `number`, and `boolean`
JavaScript has three very commonly used primitives: `string`, `number`, and `boolean`.
Each has a corresponding type in TypeScript.
As you might expect, these are the same names you'd see if you used the JavaScript `typeof` operator on a value of those types:
- `string` represents string values like `"Hello, world"`
- `number` is for numbers like `42`. JavaScript does not have a special runtime value for integers, so there's no equivalent to `int` or `float` - everything is simply `number`
- `boolean` is for the two values `true` and `false`
> The type names `String`, `Number`, and `Boolean` (starting with capital letters) are legal, but refer to some special built-in types that will very rarely appear in your code. _Always_ use `string`, `number`, or `boolean` for types.

[typescript_015] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Arrays

Sumber: ht

Arrays
To specify the type of an array like `[1, 2, 3]`, you can use the syntax `number[]`; this syntax works for any type (e.g. `string[]` is an array of strings, and so on).
You may also see this written as `Array`, which means the same thing.
We'll learn more about the syntax `T` when we cover _generics_.
> Note that `[number]` is a different thing; refer to the section on Tuples.

[typescript_016] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): `any`

Sumber: ht

`any`
TypeScript also has a special type, `any`, that you can use whenever you don't want a particular value to cause typechecking errors.
When a value is of type `any`, you can access any properties of it (which will in turn be of type `any`), call it like a function, assign it to (or from) a value of any type, or pretty much anything else that's syntactically legal:
The `any` type is useful when you don't want to write out a long type just to convince TypeScript that a particular line of

code is okay.

[typescript_017] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): `noImplicitAny`

Sumber: ht

`noImplicitAny`
When you don't specify a type, and TypeScript can't infer it from context, the compiler will typically default to `any`.
You usually want to avoid this, though, because `any` isn't type-checked.
Use the compiler flag `noImplicitAny` to flag any implicit `any` as an error.

[typescript_018] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Type Annotations on Variables

Sumber: ht

Type Annotations on Variables
When you declare a variable using `const`, `var`, or `let`, you can optionally add a type annotation to explicitly specify the type of the variable:
> TypeScript doesn't use "types on the left"-style declarations like `int x = 0;`
> Type annotations will always go _after_ the thing being typed.
In most cases, though, this isn't needed.
Wherever possible, TypeScript tries to automatically _infer_ the types in your code.
For example, the type of a variable is inferred based on the type of its initializer:
For the most part you don't need to explicitly learn the rules of inference.
If you're starting out, try using fewer type annotations than you think - you might be surprised how few you need for TypeScript to fully understand what's going on.

Contoh Kode / Perintah CLI:

```
> TypeScript doesn't use "types on the left"-style declarations like `int x = 0;`
> Type annotations will always go _after_ the thing being typed.

In most cases, though, this isn't needed.
Wherever possible, TypeScript tries to automatically _infer_ the types in your code.
For example, the type of a variable is inferred based on the type of its initializer:
```

[typescript_019] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Functions

Sumber: ht

Functions
Functions are the primary means of passing data around in JavaScript.
TypeScript allows you to specify the types of both the input and output values of functions.

[typescript_020] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Parameter Type Annotations

Sumber: ht

Parameter Type Annotations
When you declare a function, you can add type annotations after each parameter to declare what types of parameters the function accepts.
Parameter type annotations go after the parameter name:
When a parameter has a type annotation, arguments to that function will be checked:
> Even if you don't have type annotations on your parameters, TypeScript will still check that you passed the right number of arguments.

Contoh Kode / Perintah CLI:

```
When a parameter has a type annotation, arguments to that function will be checked:
```

[typescript_021] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Return Type Annotations

Sumber: ht

Return Type Annotations
You can also add return type annotations.
Return type annotations appear after the parameter list:
Much like variable type annotations, you usually don't need a return type annotation because TypeScript will infer the function's return type based on its `return` statements.
The type annotation in the above example doesn't change anything.
Some codebases will explicitly specify a return type for documentation purposes, to prevent accidental changes, or just for personal preference.
Functions Which Return Promises
If you want to annotate the return type of a function which returns a promise, you should use the `Promise` type:

Contoh Kode / Perintah CLI:

```
Much like variable type annotations, you usually don't need a return type annotation because TypeScript will infer the function's return type based on its `return` statements.
The type annotation in the above example doesn't change anything.
Some codebases will explicitly specify a return type for documentation purposes, to prevent accidental changes, or just for personal preference.

#### Functions Which Return Promises

If you want to annotate the return type of a function which returns a promise, you should use the `Promise` type:
```

[typescript_022] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Anonymous Functions

Sumber: ht

Anonymous Functions
Anonymous functions are a little bit different from function declarations.
When a function appears in a place where TypeScript can determine how it's going to be called, the parameters of that function are automatically given types.
Here's an example:
Even though the parameter `s` didn't have a type annotation, TypeScript used the types of the `forEach` function, along

with the inferred type of the array, to determine the type `s` will have.
This process is called _contextual typing_ because the _context_ that the function occurred within informs what type it should have.
Similar to the inference rules, you don't need to explicitly learn how this happens, but understanding that it _does_ happen can help you notice when type annotations aren't needed.
Later, we'll see more examples of how the context that a value occurs in can affect its type.

[typescript_023] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Object Types

Sumber: ht

Object Types
Apart from primitives, the most common sort of type you'll encounter is an _object type_.
This refers to any JavaScript value with properties, which is almost all of them!
To define an object type, we simply list its properties and their types.
For example, here's a function that takes a point-like object:
Here, we annotated the parameter with a type with two properties - `x` and `y` - which are both of type `number`.
You can use `,` or `;` to separate the properties, and the last separator is optional either way.
The type part of each property is also optional.
If you don't specify a type, it will be assumed to be `any`.

[typescript_024] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Optional Properties

Sumber: ht

Optional Properties
Object types can also specify that some or all of their properties are _optional_.
To do this, add a `?` after the property name:
In JavaScript, if you access a property that doesn't exist, you'll get the value `undefined` rather than a runtime error.
Because of this, when you _read_ from an optional property, you'll have to check for `undefined` before using it.

Contoh Kode / Perintah CLI:

```
In JavaScript, if you access a property that doesn't exist, you'll get the value `undefined` rather than a runtime error.
Because of this, when you _read_ from an optional property, you'll have to check for `undefined` before using it.
```

[typescript_025] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Union Types

Sumber: ht

Union Types
TypeScript's type system allows you to build new types out of existing ones using a large variety of operators.
Now that we know how to write a few types, it's time to start _combining_ them in interesting ways.

[typescript_026] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Defining a Union Type

Sumber: ht

Defining a Union Type
The first way to combine types you might see is a _union_ type.
A union type is a type formed from two or more other types, representing values that may be _any one_ of those types.
We refer to each of these types as the union's _members_.
Let's write a function that can operate on strings or numbers:
> The separator of the union members is allowed before the first element, so you could also write this:
>

Contoh Kode / Perintah CLI:

```
> The separator of the union members is allowed before the first element, so you could also write this:
>
```

[typescript_027] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Working with Union Types

Sumber: ht

Working with Union Types
It's easy to _provide_ a value matching a union type - simply provide a type matching any of the union's members.
If you _have_ a value of a union type, how do you work with it?
TypeScript will only allow an operation if it is valid for _every_ member of the union.
For example, if you have the union `string | number`, you can't use methods that are only available on `string`:
The solution is to _narrow_ the union with code, the same as you would in JavaScript without type annotations.
Narrowing occurs when TypeScript can deduce a more specific type for a value based on the structure of the code.
For example, TypeScript knows that only a `string` value will have a `typeof` value `"string"`:
Another example is to use a function like `Array.isArray`:
Notice that in the `else` branch, we don't need to do anything special - if `x` wasn't a `string[]`, then it must have been a `string`.
Sometimes you'll have a union where all the members have something in common.
For example, both arrays and strings have a `slice` method.
If every member in a union has a property in common, you can use that property without narrowing:
> It might be confusing that a _union_ of types appears to have the _intersection_ of those types' properties.
> This is not an accident - the name _union_ comes from type theory.
> The _union_ `number | string` is composed by taking the union _of the values_ from each type.
> Notice that given two sets with corresponding facts about each set, only the _intersection_ of those facts applies to the _union_ of the sets themselves.
> For example, if we had a room of tall people wearing hats, and another room of Spanish speakers wearing hats, after combining those rooms, the only thing we know about _every_ person is that they must be w

Contoh Kode / Perintah CLI:

```
The solution is to _narrow_ the union with code, the same as you would in JavaScript without type
annotations.
_Narrowing_ occurs when TypeScript can deduce a more specific type for a value based on the structure of
the code.

For example, TypeScript knows that only a `string` value will have a `typeof` value `"string"`:

// ----------------------------------------

Another example is to use a function like `Array.isArray`:
```

```
// ----------------------------------------

Notice that in the `else` branch, we don't need to do anything special - if `x` wasn't a `string[]`, then
it must have been a `string`.

Sometimes you'll have a union where all the members have something in common.
For example, both arrays and strings have a `slice` method.
If every member in a union has a property in common, you can use that property without narrowing:
```

[typescript_028] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Type Aliases

Sumber: ht

Type Aliases
We've been using object types and union types by writing them directly in type annotations.
This is convenient, but it's common to want to use the same type more than once and refer to it by a single name.
A _type alias_ is exactly that - a _name_ for any _type_.
The syntax for a type alias is:
You can actually use a type alias to give a name to any type at all, not just an object type.
For example, a type alias can name a union type:
Note that aliases are _only_ aliases - you cannot use type aliases to create different/distinct "versions" of the same type.
When you use the alias, it's exactly as if you had written the aliased type.
In other words, this code might _look_ illegal, but is OK according to TypeScript because both types are aliases for the same type:

Contoh Kode / Perintah CLI:

```
You can actually use a type alias to give a name to any type at all, not just an object type.
For example, a type alias can name a union type:

// ----------------------------------------

Note that aliases are _only_ aliases - you cannot use type aliases to create different/distinct
"versions" of the same type.
When you use the alias, it's exactly as if you had written the aliased type.
In other words, this code might _look_ illegal, but is OK according to TypeScript because both types are
aliases for the same type:
```

[typescript_029] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Interfaces

Sumber: ht

Interfaces
An _interface declaration_ is another way to name an object type:
Just like when we used a type alias above, the example works just as if we had used an anonymous object type.
TypeScript is only concerned with the _structure_ of the value we passed to `printCoord` - it only cares that it has the expected properties.
Being concerned only with the structure and capabilities of types is why we call TypeScript a _structurally typed_ type system.

[typescript_030] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Differences Between Type Aliases and Interfaces

Sumber: ht

Differences Between Type Aliases and Interfaces

Type aliases and interfaces are very similar, and in many cases you can choose between them freely.

Almost all features of an `interface` are available in `type`, the key distinction is that a type cannot be re-opened to add new properties vs an interface which is always extendable.

Interface

Type

Extending an interface

```
interface Animal {
name: string;
}
interface Bear extends Animal {
honey: boolean;
}
const bear = getBear();
bear.name;
bear.honey;
```

Extending a type via intersections

```
type Animal = {
name: string;
}
type Bear = Animal & {
honey: boolean;
}
const bear = getBear();
bear.name;
bear.honey;
```

Adding new fields to an existing interface

```
interface Window {
title: string;
}
interface Window {
ts: TypeScriptAPI;
}
const src = 'const a = "Hello World"';
window.ts.transpileModule(src, {});
```

A type cannot be changed after being created

```
type Window = {
title: string;
}
type Window = {
ts: TypeScriptAPI;
}
// Error: Duplicate identifier 'Window'.
```

You'll learn more about these concepts in later chapters, so don't worry if you don't understand all of these right away.

- Prior to TypeScript version 4.2, type alias names _may_ appear in error messages, sometimes in place of the equivalent anonymous type (which may or may not be desirable). Interfaces will always be named in error messages.
- Type aliases may not participate in declaration merging, but interfaces can.

- Interfaces may only be used to declare the shapes of objects, not rename primitives.
- Interface names will _always_ appear in their original form in error messages, but _only_ when they are used by name.
- Using interfaces with `extends` can often be more performant for the compiler than type aliases with intersections

For the most part, you can choose based

[typescript_031] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Type Assertions

Sumber: ht

Type Assertions

Sometimes you will have information about the type of a value that TypeScript can't know about.

For example, if you're using `document.getElementById`, TypeScript only knows that this will return _some_ kind of `HTMLElement`, but you might know that your page will always have an `HTMLCanvasElement` with a given ID.

In this situation, you can use a _type assertion_ to specify a more specific type:

Like a type annotation, type assertions are removed by the compiler and won't affect the runtime behavior of your code.

You can also use the angle-bracket syntax (except if the code is in a `.tsx` file), which is equivalent:

> Reminder: Because type assertions are removed at compile-time, there is no runtime checking associated with a type assertion.

> There won't be an exception or `null` generated if the type assertion is wrong.

TypeScript only allows type assertions which convert to a _more specific_ or _less specific_ version of a type.

This rule prevents "impossible" coercions like:

Sometimes this rule can be too conservative and will disallow more complex coercions that might be valid.

If this happens, you can use two assertions, first to `any` (or `unknown`, which we'll introduce later), then to the desired type:

Contoh Kode / Perintah CLI:

```
Like a type annotation, type assertions are removed by the compiler and won't affect the runtime behavior of your code.

You can also use the angle-bracket syntax (except if the code is in a `.tsx` file), which is equivalent:

// ---------------------------------------

> Reminder: Because type assertions are removed at compile-time, there is no runtime checking associated with a type assertion.
> There won't be an exception or `null` generated if the type assertion is wrong.

TypeScript only allows type assertions which convert to a _more specific_ or _less specific_ version of a type.
This rule prevents "impossible" coercions like:

// ---------------------------------------

Sometimes this rule can be too conservative and will disallow more complex coercions that might be valid.
If this happens, you can use two assertions, first to `any` (or `unknown`, which we'll introduce later), then to the desired type:
```

[typescript_032] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Literal Types

Sumber: ht

Literal Types
In addition to the general types `string` and `number`, we can refer to _specific_ strings and numbers in type positions.
One way to think about this is to consider how JavaScript comes with different ways to declare a variable. Both `var` and `let` allow for changing what is held inside the variable, and `const` does not. This is reflected in how TypeScript creates types for literals.
By themselves, literal types aren't very valuable:
It's not much use to have a variable that can only have one value!
But by _combining_ literals into unions, you can express a much more useful concept - for example, functions that only accept a certain set of known values:
Numeric literal types work the same way:
Of course, you can combine these with non-literal types:
There's one more kind of literal type: boolean literals.
There are only two boolean literal types, and as you might guess, they are the types `true` and `false`.
The type `boolean` itself is actually just an alias for the union `true | false`.

Contoh Kode / Perintah CLI:

```
By themselves, literal types aren't very valuable:

// ----------------------------------------

It's not much use to have a variable that can only have one value!

But by _combining_ literals into unions, you can express a much more useful concept - for example, functions that only accept a certain set of known values:

// ----------------------------------------

Numeric literal types work the same way:
```

[typescript_033] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Literal Inference

Sumber: ht

Literal Inference
When you initialize a variable with an object, TypeScript assumes that the properties of that object might change values later.
For example, if you wrote code like this:
TypeScript doesn't assume the assignment of `1` to a field which previously had `0` is an error.
Another way of saying this is that `obj.counter` must have the type `number`, not `0`, because types are used to determine both _reading_ and _writing_ behavior.
The same applies to strings:
In the above example `req.method` is inferred to be `string`, not `"GET"`. Because code can be evaluated between the creation of `req` and the call of `handleRequest` which could assign a new string like `"GUESS"` to `req.method`, TypeScript considers this code to have an error.
There are two ways to work around this.
1. You can change the inference by adding a type assertion in either location:
Change 1 means "I intend for `req.method` to always have the _literal type_ `"GET"`", preventing the possible assignment of `"GUESS"` to that field after.
Change 2 means "I know for other reasons that `req.method` has the value `"GET"`".

2. You can use `as const` to convert the entire object to be type literals:
The `as const` suffix acts like `const` but for the type system, ensuring that all properties are assigned the literal type instead of a more general version like `string` or `number`.

Contoh Kode / Perintah CLI:

```
TypeScript doesn't assume the assignment of `1` to a field which previously had `0` is an error.
Another way of saying this is that `obj.counter` must have the type `number`, not `0`, because types are used to determine both _reading_ and _writing_ behavior.

The same applies to strings:

// ----------------------------------------

In the above example `req.method` is inferred to be `string`, not `"GET"`. Because code can be evaluated between the creation of `req` and the call of `handleRequest` which could assign a new string like `"GUESS"` to `req.method`, TypeScript considers this code to have an error.

There are two ways to work around this.

1. You can change the inference by adding a type assertion in either location:

// ----------------------------------------

Change 1 means "I intend for `req.method` to always have the _literal type_ `"GET"`", preventing the possible assignment of `"GUESS"` to that field after.
   Change 2 means "I know for other reasons that `req.method` has the value `"GET"`".

2. You can use `as const` to convert the entire object to be type literals:
```

[typescript_034] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): `null` and `undefined`

Sumber: ht

`null` and `undefined`
JavaScript has two primitive values used to signal absent or uninitialized value: `null` and `undefined`.
TypeScript has two corresponding _types_ by the same names. How these types behave depends on whether you have the `strictNullChecks` option on.

[typescript_035] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): `strictNullChecks` off

Sumber: ht

`strictNullChecks` off
With `strictNullChecks` _off_, values that might be `null` or `undefined` can still be accessed normally, and the values `null` and `undefined` can be assigned to a property of any type.
This is similar to how languages without null checks (e.g. C#, Java) behave.
The lack of checking for these values tends to be a major source of bugs; we always recommend people turn `strictNullChecks` on if it's practical to do so in their codebase.

[typescript_036] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): `strictNullChecks` on

Sumber: ht

`strictNullChecks` on
With `strictNullChecks` _on_, when a value is `null` or `undefined`, you will need to test for those values before using methods or properties on that value.
Just like checking for `undefined` before using an optional property, we can use _narrowing_ to check for values that might be `null`:

[typescript_037] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Non-null Assertion Operator (Postfix `!`)

Sumber: ht

Non-null Assertion Operator (Postfix `!`)
TypeScript also has a special syntax for removing `null` and `undefined` from a type without doing any explicit checking. Writing `!` after any expression is effectively a type assertion that the value isn't `null` or `undefined`:
Just like other type assertions, this doesn't change the runtime behavior of your code, so it's important to only use `!` when you know that the value _can't_ be `null` or `undefined`.

[typescript_038] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Enums

Sumber: ht

Enums
Enums are a feature added to JavaScript by TypeScript which allows for describing a value which could be one of a set of possible named constants. Unlike most TypeScript features, this is _not_ a type-level addition to JavaScript but something added to the language and runtime. Because of this, it's a feature which you should know exists, but maybe hold off on using unless you are sure. You can read more about enums in the Enum reference page.

[typescript_039] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): Less Common Primitives

Sumber: ht

Less Common Primitives
It's worth mentioning the rest of the primitives in JavaScript which are represented in the type system.
Though we will not go into depth here.

[typescript_040] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): `bigint`

Sumber: ht

`bigint`
From ES2020 onwards, there is a primitive in JavaScript used for very large integers, `BigInt`:
You can learn more about BigInt in the TypeScript 3.2 release notes.

[typescript_041] TypeScript - Everyday Types

Everyday Types (Primitives, Arrays, any, Union Types, Type Aliases, Interfaces): `symbol`

Sumber: ht

`symbol`

There is a primitive in JavaScript used to create a globally unique reference via the function `Symbol()`:

You can learn more about them in Symbols reference page.

[typescript_042] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): Overview

Sumber: ht

Imagine we have a function called `padLeft`.

If `padding` is a `number`, it will treat that as the number of spaces we want to prepend to `input`.

If `padding` is a `string`, it should just prepend `padding` to `input`.

Let's try to implement the logic for when `padLeft` is passed a `number` for `padding`.

Uh-oh, we're getting an error on `padding`.

TypeScript is warning us that we're passing a value with type `number | string` to the `repeat` function, which only accepts a `number`, and it's right.

In other words, we haven't explicitly checked if `padding` is a `number` first, nor are we handling the case where it's a `string`, so let's do exactly that.

If this mostly looks like uninteresting JavaScript code, that's sort of the point.

Apart from the annotations we put in place, this TypeScript code looks like JavaScript.

The idea is that TypeScript's type system aims to make it as easy as possible to write typical JavaScript code without bending over backwards to get type safety.

While it might not look like much, there's actually a lot going on under the covers here.

Much like how TypeScript analyzes runtime values using static types, it overlays type analysis on JavaScript's runtime control flow constructs like `if/else`, conditional ternaries, loops, truthiness checks, etc., which can all affect those types.

Within our `if` check, TypeScript sees `typeof padding === "number"` and understands that as a special form of code called a _type guard_.

TypeScript follows possible paths of execution that our programs can take to analyze the most specific possible type of a value at a given position.

It looks at these special checks (called _type guards_) and assignments, and the process of refining types to more specific types than declared is called _narrowing_.

In many edi

Contoh Kode / Perintah CLI:

```
If `padding` is a `number`, it will treat that as the number of spaces we want to prepend to `input`.
If `padding` is a `string`, it should just prepend `padding` to `input`.
Let's try to implement the logic for when `padLeft` is passed a `number` for `padding`.

// ----------------------------------------

Uh-oh, we're getting an error on `padding`.
TypeScript is warning us that we're passing a value with type `number | string` to the `repeat` function, which only accepts a `number`, and it's right.
In other words, we haven't explicitly checked if `padding` is a `number` first, nor are we handling the case where it's a `string`, so let's do exactly that.

// ----------------------------------------

If this mostly looks like uninteresting JavaScript code, that's sort of the point.
Apart from the annotations we put in place, this TypeScript code looks like JavaScript.
The idea is that TypeScript's type system aims to make it as easy as possible to write typical JavaScript code without bending over backwards to get type safety.
```

```
While it might not look like much, there's actually a lot going on under the covers here.
Much like how TypeScript analyzes runtime values using static types, it overlays type analysis on JavaScript's runtime control flow constructs like `if/else`, conditional ternaries, loops, truthiness checks, etc., which can all affect those types.

Within our `if` check, TypeScript sees `typeof padding === "number"` and understands that as a special form of code called a _type guard_.
TypeScript follows possible paths of execution that our programs can take to analyze the most specific possible type of a value at a given position.
It looks at these special checks (called _type guards_) and assignments, and the process of refining types to more specific types than declared is called _narrowing_.
In many editors we can observe these types as they change, and we'll even do so in our examples.
```

[typescript_043] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): `typeof` type guards

Sumber: ht

`typeof` type guards
As we've seen, JavaScript supports a `typeof` operator which can give very basic information about the type of values we have at runtime.
TypeScript expects this to return a certain set of strings:
- `"string"`
- `"number"`
- `"bigint"`
- `"boolean"`
- `"symbol"`
- `"undefined"`
- `"object"`
- `"function"`
Like we saw with `padLeft`, this operator comes up pretty often in a number of JavaScript libraries, and TypeScript can understand it to narrow types in different branches.
In TypeScript, checking against the value returned by `typeof` is a type guard.
Because TypeScript encodes how `typeof` operates on different values, it knows about some of its quirks in JavaScript.
For example, notice that in the list above, `typeof` doesn't return the string `null`.
Check out the following example:
In the `printAll` function, we try to check if `strs` is an object to see if it's an array type (now might be a good time to reinforce that arrays are object types in JavaScript).
But it turns out that in JavaScript, `typeof null` is actually `"object"`!
This is one of those unfortunate accidents of history.
Users with enough experience might not be surprised, but not everyone has run into this in JavaScript; luckily, TypeScript lets us know that `strs` was only narrowed down to `string[] | null` instead of just `string[]`.
This might be a good segue into what we'll call "truthiness" checking.

[typescript_044] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): Truthiness narrowing

Sumber: ht

Truthiness narrowing
Truthiness might not be a word you'll find in the dictionary, but it's very much something you'll hear about in JavaScript.
In JavaScript, we can use any expression in conditionals, `&&`s, `||`s, `if` statements, Boolean negations (`!`), and more.
As an example, `if` statements don't expect their condition to always have the type `boolean`.

In JavaScript, constructs like `if` first "coerce" their conditions to `boolean`s to make sense of them, and then choose their branches depending on whether the result is `true` or `false`.
Values like
- `0`
- `NaN`
- `""` (the empty string)
- `0n` (the `bigint` version of zero)
- `null`
- `undefined`

all coerce to `false`, and other values get coerced to `true`.
You can always coerce values to `boolean`s by running them through the `Boolean` function, or by using the shorter double-Boolean negation. (The latter has the advantage that TypeScript infers a narrow literal boolean type `true`, while inferring the first as type `boolean`.)
It's fairly popular to leverage this behavior, especially for guarding against values like `null` or `undefined`.
As an example, let's try using it for our `printAll` function.
You'll notice that we've gotten rid of the error above by checking if `strs` is truthy.
This at least prevents us from dreaded errors when we run our code like:
Keep in mind though that truthiness checking on primitives can often be error prone.
As an example, consider a different attempt at writing `printAll`
We wrapped the entire body of the function in a truthy check, but this has a subtle downside: we may no longer be handling the empty string case correctly.
TypeScript doesn't hurt us here at all, but this behavior is worth noting if you're less familiar with JavaScript.
TypeScript can often help you

Contoh Kode / Perintah CLI:

```
In JavaScript, constructs like `if` first "coerce" their conditions to `boolean`s to make sense of them, and then choose their branches depending on whether the result is `true` or `false`.
Values like

- `0`
- `NaN`
- `""` (the empty string)
- `0n` (the `bigint` version of zero)
- `null`
- `undefined`

all coerce to `false`, and other values get coerced to `true`.
You can always coerce values to `boolean`s by running them through the `Boolean` function, or by using the shorter double-Boolean negation. (The latter has the advantage that TypeScript infers a narrow literal boolean type `true`, while inferring the first as type `boolean`.)

// ----------------------------------------

It's fairly popular to leverage this behavior, especially for guarding against values like `null` or `undefined`.
As an example, let's try using it for our `printAll` function.

// ----------------------------------------

You'll notice that we've gotten rid of the error above by checking if `strs` is truthy.
This at least prevents us from dreaded errors when we run our code like:
```

[typescript_045] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): Equality narrowing

Sumber: ht

Equality narrowing
TypeScript also uses `switch` statements and equality checks like `===`, `!==`, `==`, and `!=` to narrow types.
For example:
When we checked that `x` and `y` are both equal in the above example, TypeScript knew their types also had to be equal.
Since `string` is the only common type that both `x` and `y` could take on, TypeScript knows that `x` and `y` must be `string`s in the first branch.
Checking against specific literal values (as opposed to variables) works also.
In our section about truthiness narrowing, we wrote a `printAll` function which was error-prone because it accidentally didn't handle empty strings properly.
Instead we could have done a specific check to block out `null`s, and TypeScript still correctly removes `null` from the type of `strs`.
JavaScript's looser equality checks with `==` and `!=` also get narrowed correctly.
If you're unfamiliar, checking whether something `== null` actually not only checks whether it is specifically the value `null` - it also checks whether it's potentially `undefined`.
The same applies to `== undefined`: it checks whether a value is either `null` or `undefined`.

Contoh Kode / Perintah CLI:

```
When we checked that `x` and `y` are both equal in the above example, TypeScript knew their types also had to be equal.
Since `string` is the only common type that both `x` and `y` could take on, TypeScript knows that `x` and `y` must be `string`s in the first branch.

Checking against specific literal values (as opposed to variables) works also.
In our section about truthiness narrowing, we wrote a `printAll` function which was error-prone because it accidentally didn't handle empty strings properly.
Instead we could have done a specific check to block out `null`s, and TypeScript still correctly removes `null` from the type of `strs`.

// ----------------------------------------

JavaScript's looser equality checks with `==` and `!=` also get narrowed correctly.
If you're unfamiliar, checking whether something `== null` actually not only checks whether it is specifically the value `null` - it also checks whether it's potentially `undefined`.
The same applies to `== undefined`: it checks whether a value is either `null` or `undefined`.
```

[typescript_046] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): The `in` operator narrowing

Sumber: ht

The `in` operator narrowing
JavaScript has an operator for determining if an object or its prototype chain has a property with a name: the `in` operator.
TypeScript takes this into account as a way to narrow down potential types.
For example, with the code: `"value" in x` where `"value"` is a string literal and `x` is a union type.
The "true" branch narrows `x`'s types which have either an optional or required property `value`, and the "false" branch narrows to types which have an optional or missing property `value`.
To reiterate, optional properties will exist in both sides for narrowing. For example, a human could both swim and fly (with the right equipment) and thus should show up in both sides of the `in` check:

Contoh Kode / Perintah CLI:

```
To reiterate, optional properties will exist in both sides for narrowing. For example, a human could both swim and fly (with the right equipment) and thus should show up in both sides of the `in` check:

<!-- prettier-ignore -->
```

[typescript_047] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): `instanceof` narrowing

Sumber: ht

`instanceof` narrowing
JavaScript has an operator for checking whether or not a value is an "instance" of another value.
More specifically, in JavaScript `x instanceof Foo` checks whether the _prototype chain_ of `x` contains `Foo.prototype`.
While we won't dive deep here, and you'll see more of this when we get into classes, they can still be useful for most values that can be constructed with `new`.
As you might have guessed, `instanceof` is also a type guard, and TypeScript narrows in branches guarded by `instanceof`s.

[typescript_048] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): Assignments

Sumber: ht

Assignments
As we mentioned earlier, when we assign to any variable, TypeScript looks at the right side of the assignment and narrows the left side appropriately.
Notice that each of these assignments is valid.
Even though the observed type of `x` changed to `number` after our first assignment, we were still able to assign a `string` to `x`.
This is because the _declared type_ of `x` - the type that `x` started with - is `string | number`, and assignability is always checked against the declared type.
If we'd assigned a `boolean` to `x`, we'd have seen an error since that wasn't part of the declared type.

Contoh Kode / Perintah CLI:

```
Notice that each of these assignments is valid.
Even though the observed type of `x` changed to `number` after our first assignment, we were still able to assign a `string` to `x`.
This is because the _declared type_ of `x` - the type that `x` started with - is `string | number`, and assignability is always checked against the declared type.

If we'd assigned a `boolean` to `x`, we'd have seen an error since that wasn't part of the declared type.
```

[typescript_049] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): Control flow analysis

Sumber: ht

Control flow analysis
Up until this point, we've gone through some basic examples of how TypeScript narrows within specific branches.
But there's a bit more going on than just walking up from every variable and looking for type guards in `if`s, `while`s, conditionals, etc.
For example
`padLeft` returns from within its first `if` block.
TypeScript was able to analyze this code and see that the rest of the body (`return padding + input;`) is _unreachable_ in the case where `padding` is a `number`.

As a result, it was able to remove `number` from the type of `padding` (narrowing from `string | number` to `string`) for the rest of the function.
This analysis of code based on reachability is called _control flow analysis_, and TypeScript uses this flow analysis to narrow types as it encounters type guards and assignments.
When a variable is analyzed, control flow can split off and re-merge over and over again, and that variable can be observed to have a different type at each point.

Contoh Kode / Perintah CLI:

```
`padLeft` returns from within its first `if` block.
TypeScript was able to analyze this code and see that the rest of the body (`return padding + input;`) is
_unreachable_ in the case where `padding` is a `number`.
As a result, it was able to remove `number` from the type of `padding` (narrowing from `string | number`
to `string`) for the rest of the function.

This analysis of code based on reachability is called _control flow analysis_, and TypeScript uses this
flow analysis to narrow types as it encounters type guards and assignments.
When a variable is analyzed, control flow can split off and re-merge over and over again, and that
variable can be observed to have a different type at each point.
```

[typescript_050] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): Using type predicates

Sumber: ht

Using type predicates
We've worked with existing JavaScript constructs to handle narrowing so far, however sometimes you want more direct control over how types change throughout your code.
To define a user-defined type guard, we simply need to define a function whose return type is a _type predicate_:
`pet is Fish` is our type predicate in this example.
A predicate takes the form `parameterName is Type`, where `parameterName` must be the name of a parameter from the current function signature.
Any time `isFish` is called with some variable, TypeScript will _narrow_ that variable to that specific type if the original type is compatible.
Notice that TypeScript not only knows that `pet` is a `Fish` in the `if` branch;
it also knows that in the `else` branch, you _don't_ have a `Fish`, so you must have a `Bird`.
You may use the type guard `isFish` to filter an array of `Fish | Bird` and obtain an array of `Fish`:
In addition, classes can use `this is Type` to narrow their type.

Contoh Kode / Perintah CLI:

```
`pet is Fish` is our type predicate in this example.
A predicate takes the form `parameterName is Type`, where `parameterName` must be the name of a parameter
from the current function signature.

Any time `isFish` is called with some variable, TypeScript will _narrow_ that variable to that specific
type if the original type is compatible.

// ----------------------------------------

Notice that TypeScript not only knows that `pet` is a `Fish` in the `if` branch;
it also knows that in the `else` branch, you _don't_ have a `Fish`, so you must have a `Bird`.

You may use the type guard `isFish` to filter an array of `Fish | Bird` and obtain an array of `Fish`:
```

[typescript_051] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): Assertion functions

Sumber: ht

Assertion functions
Types can also be narrowed using Assertion functions.

[typescript_052] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): Discriminated unions

Sumber: ht

Discriminated unions
Most of the examples we've looked at so far have focused around narrowing single variables with simple types like `string`, `boolean`, and `number`.
While this is common, most of the time in JavaScript we'll be dealing with slightly more complex structures.
For some motivation, let's imagine we're trying to encode shapes like circles and squares.
Circles keep track of their radiuses and squares keep track of their side lengths.
We'll use a field called `kind` to tell which shape we're dealing with.
Here's a first attempt at defining `Shape`.
Notice we're using a union of string literal types: `"circle"` and `"square"` to tell us whether we should treat the shape as a circle or square respectively.
By using `"circle" | "square"` instead of `string`, we can avoid misspelling issues.
We can write a `getArea` function that applies the right logic based on if it's dealing with a circle or square.
We'll first try dealing with circles.
Under `strictNullChecks` that gives us an error - which is appropriate since `radius` might not be defined.
But what if we perform the appropriate checks on the `kind` property?
Hmm, TypeScript still doesn't know what to do here.
We've hit a point where we know more about our values than the type checker does.
We could try to use a non-null assertion (a `!` after `shape.radius`) to say that `radius` is definitely present.
But this doesn't feel ideal.
We had to shout a bit at the type-checker with those non-null assertions (`!`) to convince it that `shape.radius` was defined, but those assertions are error-prone if we start to move code around.
Additionally, outside of `strictNullChecks` we're able to accidentally access any of those fields anyway (since optional properties are just assumed to always be present when readin

Contoh Kode / Perintah CLI:

```
Notice we're using a union of string literal types: `"circle"` and `"square"` to tell us whether we should treat the shape as a circle or square respectively.
By using `"circle" | "square"` instead of `string`, we can avoid misspelling issues.

// ----------------------------------------

We can write a `getArea` function that applies the right logic based on if it's dealing with a circle or square.
We'll first try dealing with circles.

// ----------------------------------------

<!-- TODO -->

Under [`strictNullChecks`](/tsconfig#strictNullChecks) that gives us an error - which is appropriate since `radius` might not be defined.
But what if we perform the appropriate checks on the `kind` property?
```

[typescript_053] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): The `never` type

Sumber: ht

The `never` type
When narrowing, you can reduce the options of a union to a point where you have removed all possibilities and have nothing left.
In those cases, TypeScript will use a `never` type to represent a state which shouldn't exist.

[typescript_054] TypeScript - Narrowing & Type Guards

Narrowing (typeof, Truthiness, Equality, in operator, Type Predicates): Exhaustiveness checking

Sumber: ht

Exhaustiveness checking
The `never` type is assignable to every type; however, no type is assignable to `never` (except `never` itself). This means you can use narrowing and rely on `never` turning up to do exhaustive checking in a `switch` statement.
For example, adding a `default` to our `getArea` function which tries to assign the shape to `never` will not raise an error when every possible case has been handled.
Adding a new member to the `Shape` union, will cause a TypeScript error:

Contoh Kode / Perintah CLI:

```
Adding a new member to the `Shape` union, will cause a TypeScript error:
```

[typescript_055] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Overview

Sumber: ht

Functions are the basic building block of any application, whether they're local functions, imported from another module, or methods on a class.
They're also values, and just like other values, TypeScript has many ways to describe how functions can be called.
Let's learn about how to write types that describe functions.

[typescript_056] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Function Type Expressions

Sumber: ht

Function Type Expressions
The simplest way to describe a function is with a _function type expression_.
These types are syntactically similar to arrow functions:
The syntax `(a: string) => void` means "a function with one parameter, named `a`, of type `string`, that doesn't have a return value".
Just like with function declarations, if a parameter type isn't specified, it's implicitly `any`.
> Note that the parameter name is **required**. The function type `(string) => void` means "a function with a parameter named `string` of type `any`"!
Of course, we can use a type alias to name a function type:

Contoh Kode / Perintah CLI:

```
The syntax `(a: string) => void` means "a function with one parameter, named `a`, of type `string`, that doesn't have a return value".
Just like with function declarations, if a parameter type isn't specified, it's implicitly `any`.

> Note that the parameter name is **required**. The function type `(string) => void` means "a function
```

```
with a parameter named `string` of type `any`"!

Of course, we can use a type alias to name a function type:
```

[typescript_057] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Call Signatures

Sumber: ht

Call Signatures
In JavaScript, functions can have properties in addition to being callable.
However, the function type expression syntax doesn't allow for declaring properties.
If we want to describe something callable with properties, we can write a _call signature_ in an object type:
Note that the syntax is slightly different compared to a function type expression - use `:` between the parameter list and the return type rather than `=>`.

[typescript_058] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Construct Signatures

Sumber: ht

Construct Signatures
JavaScript functions can also be invoked with the `new` operator.
TypeScript refers to these as _constructors_ because they usually create a new object.
You can write a _construct signature_ by adding the `new` keyword in front of a call signature:
Some objects, like JavaScript's `Date` object, can be called with or without `new`.
You can combine call and construct signatures in the same type arbitrarily:

Contoh Kode / Perintah CLI:

```
Some objects, like JavaScript's `Date` object, can be called with or without `new`.
You can combine call and construct signatures in the same type arbitrarily:
```

[typescript_059] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Generic Functions

Sumber: ht

Generic Functions
It's common to write a function where the types of the input relate to the type of the output, or where the types of two inputs are related in some way.
Let's consider for a moment a function that returns the first element of an array:
This function does its job, but unfortunately has the return type `any`.
It'd be better if the function returned the type of the array element.
In TypeScript, _generics_ are used when we want to describe a correspondence between two values.
We do this by declaring a _type parameter_ in the function signature:
By adding a type parameter `Type` to this function and using it in two places, we've created a link between the input of the function (the array) and the output (the return value).
Now when we call it, a more specific type comes out:

Contoh Kode / Perintah CLI:

```
This function does its job, but unfortunately has the return type `any`.
It'd be better if the function returned the type of the array element.

In TypeScript, _generics_ are used when we want to describe a correspondence between two values.
We do this by declaring a _type parameter_ in the function signature:
```

```
// ----------------------------------------

By adding a type parameter `Type` to this function and using it in two places, we've created a link
between the input of the function (the array) and the output (the return value).
Now when we call it, a more specific type comes out:
```

[typescript_060] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Inference

Sumber: ht

Inference
Note that we didn't have to specify `Type` in this sample.
The type was _inferred_ - chosen automatically - by TypeScript.
We can use multiple type parameters as well.
For example, a standalone version of `map` would look like this:
Note that in this example, TypeScript could infer both the type of the `Input` type parameter (from the given `string` array), as well as the `Output` type parameter based on the return value of the function expression (`number`).

[typescript_061] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Constraints

Sumber: ht

Constraints
We've written some generic functions that can work on _any_ kind of value.
Sometimes we want to relate two values, but can only operate on a certain subset of values.
In this case, we can use a _constraint_ to limit the kinds of types that a type parameter can accept.
Let's write a function that returns the longer of two values.
To do this, we need a `length` property that's a number.
We _constrain_ the type parameter to that type by writing an `extends` clause:
There are a few interesting things to note in this example.
We allowed TypeScript to _infer_ the return type of `longest`.
Return type inference also works on generic functions.
Because we constrained `Type` to `{ length: number }`, we were allowed to access the `.length` property of the `a` and `b` parameters.
Without the type constraint, we wouldn't be able to access those properties because the values might have been some other type without a length property.
The types of `longerArray` and `longerString` were inferred based on the arguments.
Remember, generics are all about relating two or more values with the same type!
Finally, just as we'd like, the call to `longest(10, 100)` is rejected because the `number` type doesn't have a `.length` property.

[typescript_062] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Working with Constrained Values

Sumber: ht

Working with Constrained Values
Here's a common error when working with generic constraints:
It might look like this function is OK - `Type` is constrained to `{ length: number }`, and the function either returns `Type` or a value matching that constraint.

The problem is that the function promises to return the _same_ kind of object as was passed in, not just _some_ object matching the constraint.
If this code were legal, you could write code that definitely wouldn't work:

Contoh Kode / Perintah CLI:

```
It might look like this function is OK - `Type` is constrained to `{ length: number }`, and the function either returns `Type` or a value matching that constraint.
The problem is that the function promises to return the _same_ kind of object as was passed in, not just _some_ object matching the constraint.
If this code were legal, you could write code that definitely wouldn't work:
```

[typescript_063] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Specifying Type Arguments

Sumber: ht

Specifying Type Arguments
TypeScript can usually infer the intended type arguments in a generic call, but not always.
For example, let's say you wrote a function to combine two arrays:
Normally it would be an error to call this function with mismatched arrays:
If you intended to do this, however, you could manually specify `Type`:

Contoh Kode / Perintah CLI:

```
Normally it would be an error to call this function with mismatched arrays:

// ---------------------------------------

If you intended to do this, however, you could manually specify `Type`:
```

[typescript_064] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Guidelines for Writing Good Generic Functions

Sumber: ht

Guidelines for Writing Good Generic Functions
Writing generic functions is fun, and it can be easy to get carried away with type parameters.
Having too many type parameters or using constraints where they aren't needed can make inference less successful, frustrating callers of your function.
Push Type Parameters Down
Here are two ways of writing a function that appear similar:
These might seem identical at first glance, but `firstElement1` is a much better way to write this function.
Its inferred return type is `Type`, but `firstElement2`'s inferred return type is `any` because TypeScript has to resolve the `arr[0]` expression using the constraint type, rather than "waiting" to resolve the element during a call.
> **Rule**: When possible, use the type parameter itself rather than constraining it
Use Fewer Type Parameters
Here's another pair of similar functions:
We've created a type parameter `Func` that _doesn't relate two values_.
That's always a red flag, because it means callers wanting to specify type arguments have to manually specify an extra type argument for no reason.
`Func` doesn't do anything but make the function harder to read and reason about!
> **Rule**: Always use as few type parameters as possible
Type Parameters Should Appear Twice
Sometimes we forget that a function might not need to be generic:
We could just as easily have written a simpler version:
Remember, type parameters are for _relating the types of multiple values_.

If a type parameter is only used once in the function signature, it's not relating anything.
This includes the inferred return type; for example, if `Str` was part of the inferred return type of `greet`, it would be relating the argument and return types, so would be used _twice_ despite appearing only once in t

Contoh Kode / Perintah CLI:

```
These might seem identical at first glance, but `firstElement1` is a much better way to write this function.
Its inferred return type is `Type`, but `firstElement2`'s inferred return type is `any` because TypeScript has to resolve the `arr[0]` expression using the constraint type, rather than "waiting" to resolve the element during a call.

> **Rule**: When possible, use the type parameter itself rather than constraining it

#### Use Fewer Type Parameters

Here's another pair of similar functions:

// ----------------------------------------

We've created a type parameter `Func` that _doesn't relate two values_.
That's always a red flag, because it means callers wanting to specify type arguments have to manually specify an extra type argument for no reason.
`Func` doesn't do anything but make the function harder to read and reason about!

> **Rule**: Always use as few type parameters as possible

#### Type Parameters Should Appear Twice

Sometimes we forget that a function might not need to be generic:

// ----------------------------------------

We could just as easily have written a simpler version:
```

[typescript_065] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Optional Parameters

Sumber: ht

Optional Parameters
Functions in JavaScript often take a variable number of arguments.
For example, the `toFixed` method of `number` takes an optional digit count:
We can model this in TypeScript by marking the parameter as _optional_ with `?`:
Although the parameter is specified as type `number`, the `x` parameter will actually have the type `number | undefined` because unspecified parameters in JavaScript get the value `undefined`.
You can also provide a parameter _default_:
Now in the body of `f`, `x` will have type `number` because any `undefined` argument will be replaced with `10`.
Note that when a parameter is optional, callers can always pass `undefined`, as this simply simulates a "missing" argument:

Contoh Kode / Perintah CLI:

```
We can model this in TypeScript by marking the parameter as _optional_ with `?`:

// ----------------------------------------

Although the parameter is specified as type `number`, the `x` parameter will actually have the type `number | undefined` because unspecified parameters in JavaScript get the value `undefined`.

You can also provide a parameter _default_:
```

```
// ----------------------------------------

Now in the body of `f`, `x` will have type `number` because any `undefined` argument will be replaced
with `10`.
Note that when a parameter is optional, callers can always pass `undefined`, as this simply simulates a
"missing" argument:
```

[typescript_066] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Optional Parameters in Callbacks

Sumber: ht

Optional Parameters in Callbacks

Once you've learned about optional parameters and function type expressions, it's very easy to make the following mistakes when writing functions that invoke callbacks:

What people usually intend when writing `index?` as an optional parameter is that they want both of these calls to be legal:

What this _actually_ means is that _`callback` might get invoked with one argument_.

In other words, the function definition says that the implementation might look like this:

In turn, TypeScript will enforce this meaning and issue errors that aren't really possible:

In JavaScript, if you call a function with more arguments than there are parameters, the extra arguments are simply ignored.

TypeScript behaves the same way.

Functions with fewer parameters (of the same types) can always take the place of functions with more parameters.

> **Rule**: When writing a function type for a callback, _never_ write an optional parameter unless you intend to _call_ the function without passing that argument

Contoh Kode / Perintah CLI:

```
What people usually intend when writing `index?` as an optional parameter is that they want both of these
calls to be legal:

// ----------------------------------------

What this _actually_ means is that _`callback` might get invoked with one argument_.
In other words, the function definition says that the implementation might look like this:

// ----------------------------------------

In turn, TypeScript will enforce this meaning and issue errors that aren't really possible:

<!-- prettier-ignore -->
```

[typescript_067] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Function Overloads

Sumber: ht

Function Overloads

Some JavaScript functions can be called in a variety of argument counts and types.

For example, you might write a function to produce a `Date` that takes either a timestamp (one argument) or a month/day/year specification (three arguments).

In TypeScript, we can specify a function that can be called in different ways by writing _overload signatures_.

To do this, write some number of function signatures (usually two or more), followed by the body of the function:

In this example, we wrote two overloads: one accepting one argument, and another accepting three arguments.

These first two signatures are called the _overload signatures_.

Then, we wrote a function implementation with a compatible signature.
Functions have an _implementation_ signature, but this signature can't be called directly.
Even though we wrote a function with two optional parameters after the required one, it can't be called with two parameters!

[typescript_068] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Overload Signatures and the Implementation Signature

Sumber: ht

Overload Signatures and the Implementation Signature
This is a common source of confusion.
Often people will write code like this and not understand why there is an error:
Again, the signature used to write the function body can't be "seen" from the outside.
> The signature of the _implementation_ is not visible from the outside.
> When writing an overloaded function, you should always have _two_ or more signatures above the implementation of the function.
The implementation signature must also be _compatible_ with the overload signatures.
For example, these functions have errors because the implementation signature doesn't match the overloads in a correct way:

Contoh Kode / Perintah CLI:

```
Again, the signature used to write the function body can't be "seen" from the outside.

> The signature of the _implementation_ is not visible from the outside.
> When writing an overloaded function, you should always have _two_ or more signatures above the implementation of the function.

The implementation signature must also be _compatible_ with the overload signatures.
For example, these functions have errors because the implementation signature doesn't match the overloads in a correct way:
```

[typescript_069] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Writing Good Overloads

Sumber: ht

Writing Good Overloads
Like generics, there are a few guidelines you should follow when using function overloads.
Following these principles will make your function easier to call, easier to understand, and easier to implement.
Let's consider a function that returns the length of a string or an array:
This function is fine; we can invoke it with strings or arrays.
However, we can't invoke it with a value that might be a string _or_ an array, because TypeScript can only resolve a function call to a single overload:
Because both overloads have the same argument count and same return type, we can instead write a non-overloaded version of the function:
This is much better!
Callers can invoke this with either sort of value, and as an added bonus, we don't have to figure out a correct implementation signature.
> Always prefer parameters with union types instead of overloads when possible

Contoh Kode / Perintah CLI:

```
This function is fine; we can invoke it with strings or arrays.
However, we can't invoke it with a value that might be a string _or_ an array, because TypeScript can only resolve a function call to a single overload:
```

```
// ----------------------------------------

Because both overloads have the same argument count and same return type, we can instead write a non-overloaded version of the function:
```

[typescript_070] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Declaring `this` in a Function

Sumber: ht

Declaring `this` in a Function
TypeScript will infer what the `this` should be in a function via code flow analysis, for example in the following:
TypeScript understands that the function `user.becomeAdmin` has a corresponding `this` which is the outer object `user`. `this`, _heh_, can be enough for a lot of cases, but there are a lot of cases where you need more control over what object `this` represents. The JavaScript specification states that you cannot have a parameter called `this`, and so TypeScript uses that syntax space to let you declare the type for `this` in the function body.
This pattern is common with callback-style APIs, where another object typically controls when your function is called. Note that you need to use `function` and not arrow functions to get this behavior:

Contoh Kode / Perintah CLI:

```
TypeScript understands that the function `user.becomeAdmin` has a corresponding `this` which is the outer object `user`. `this`, _heh_, can be enough for a lot of cases, but there are a lot of cases where you need more control over what object `this` represents. The JavaScript specification states that you cannot have a parameter called `this`, and so TypeScript uses that syntax space to let you declare the type for `this` in the function body.

// ----------------------------------------

This pattern is common with callback-style APIs, where another object typically controls when your function is called. Note that you need to use `function` and not arrow functions to get this behavior:
```

[typescript_071] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Other Types to Know About

Sumber: ht

Other Types to Know About
There are some additional types you'll want to recognize that appear often when working with function types.
Like all types, you can use them everywhere, but these are especially relevant in the context of functions.

[typescript_072] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): `void`

Sumber: ht

`void`
`void` represents the return value of functions which don't return a value.
It's the inferred type any time a function doesn't have any `return` statements, or doesn't return any explicit value from those return statements:
In JavaScript, a function that doesn't return any value will implicitly return the value `undefined`.
However, `void` and `undefined` are not the same thing in TypeScript.
There are further details at the end of this chapter.
> `void` is not the same as `undefined`.

[typescript_073] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): `object`

Sumber: ht

`object`
The special type `object` refers to any value that isn't a primitive (`string`, `number`, `bigint`, `boolean`, `symbol`, `null`, or `undefined`).
This is different from the _empty object type_ `{}`, and also different from the global type `Object`.
It's very likely you will never use `Object`.
> `object` is not `Object`. **Always** use `object`!
Note that in JavaScript, function values are objects: They have properties, have `Object.prototype` in their prototype chain, are `instanceof Object`, you can call `Object.keys` on them, and so on.
For this reason, function types are considered to be `object`s in TypeScript.

[typescript_074] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): `unknown`

Sumber: ht

`unknown`
The `unknown` type represents _any_ value.
This is similar to the `any` type, but is safer because it's not legal to do anything with an `unknown` value:
This is useful when describing function types because you can describe functions that accept any value without having `any` values in your function body.
Conversely, you can describe a function that returns a value of unknown type:

Contoh Kode / Perintah CLI:

```
This is useful when describing function types because you can describe functions that accept any value without having `any` values in your function body.

Conversely, you can describe a function that returns a value of unknown type:
```

[typescript_075] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): `never`

Sumber: ht

`never`
Some functions _never_ return a value:
The `never` type represents values which are _never_ observed.
In a return type, this means that the function throws an exception or terminates execution of the program.
`never` also appears when TypeScript determines there's nothing left in a union.

Contoh Kode / Perintah CLI:

```
The `never` type represents values which are _never_ observed.
In a return type, this means that the function throws an exception or terminates execution of the program.

`never` also appears when TypeScript determines there's nothing left in a union.
```

[typescript_076] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): `Function`

Sumber: ht

`Function`
The global type `Function` describes properties like `bind`, `call`, `apply`, and others present on all function values in JavaScript.
It also has the special property that values of type `Function` can always be called; these calls return `any`:
This is an _untyped function call_ and is generally best avoided because of the unsafe `any` return type.
If you need to accept an arbitrary function but don't intend to call it, the type `() => void` is generally safer.

[typescript_077] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Rest Parameters and Arguments

Sumber: ht

Rest Parameters and Arguments
Background Reading:
Rest Parameters
Spread Syntax

[typescript_078] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Rest Parameters

Sumber: ht

Rest Parameters
In addition to using optional parameters or overloads to make functions that can accept a variety of fixed argument counts, we can also define functions that take an _unbounded_ number of arguments using _rest parameters_.
A rest parameter appears after all other parameters, and uses the `...` syntax:
In TypeScript, the type annotation on these parameters is implicitly `any[]` instead of `any`, and any type annotation given must be of the form `Array` or `T[]`, or a tuple type (which we'll learn about later).

[typescript_079] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Rest Arguments

Sumber: ht

Rest Arguments
Conversely, we can _provide_ a variable number of arguments from an iterable object (for example, an array) using the spread syntax.
For example, the `push` method of arrays takes any number of arguments:
Note that in general, TypeScript does not assume that arrays are immutable.
This can lead to some surprising behavior:
The best fix for this situation depends a bit on your code, but in general a `const` context is the most straightforward solution:
Using rest arguments may require turning on `downlevelIteration` when targeting older runtimes.

Contoh Kode / Perintah CLI:

```
Note that in general, TypeScript does not assume that arrays are immutable.
This can lead to some surprising behavior:

// ----------------------------------------
```

```
The best fix for this situation depends a bit on your code, but in general a `const` context is the most straightforward solution:
```

[typescript_080] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Parameter Destructuring

Sumber: ht

Parameter Destructuring
Background Reading:
Destructuring Assignment
You can use parameter destructuring to conveniently unpack objects provided as an argument into one or more local variables in the function body.
In JavaScript, it looks like this:
The type annotation for the object goes after the destructuring syntax:
This can look a bit verbose, but you can use a named type here as well:

Contoh Kode / Perintah CLI:

```
function sum({ a, b, c }) {
  console.log(a + b + c);
}
sum({ a: 10, b: 3, c: 9 });

// ----------------------------------------

This can look a bit verbose, but you can use a named type here as well:
```

[typescript_081] TypeScript - Functions & Call Signatures

More on Functions (Generic Functions, Optional Parameters, Function Overloads): Return type `void`

Sumber: ht

Return type `void`
The `void` return type for functions can produce some unusual, but expected behavior.
Contextual typing with a return type of `void` does **not** force functions to **not** return something. Another way to say this is a contextual function type with a `void` return type (`type voidFunc = () => void`), when implemented, can return _any_ other value, but it will be ignored.
Thus, the following implementations of the type `() => void` are valid:
And when the return value of one of these functions is assigned to another variable, it will retain the type of `void`:
This behavior exists so that the following code is valid even though `Array.prototype.push` returns a number and the `Array.prototype.forEach` method expects a function with a return type of `void`.
There is one other special case to be aware of, when a literal function definition has a `void` return type, that function must **not** return anything.
For more on `void` please refer to these other documentation entries:
- FAQ - "Why are functions returning non-void assignable to function returning void?"

Contoh Kode / Perintah CLI:

```
And when the return value of one of these functions is assigned to another variable, it will retain the type of `void`:

// ----------------------------------------

This behavior exists so that the following code is valid even though `Array.prototype.push` returns a number and the `Array.prototype.forEach` method expects a function with a return type of `void`.

// ----------------------------------------
```

```
There is one other special case to be aware of, when a literal function definition has a `void` return type, that function must **not** return anything.
```

[typescript_082] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Overview

Sumber: ht

In JavaScript, the fundamental way that we group and pass around data is through objects.
In TypeScript, we represent those through _object types_.
As we've seen, they can be anonymous:
or they can be named by using either an interface:
or a type alias:
In all three examples above, we've written functions that take objects that contain the property `name` (which must be a `string`) and `age` (which must be a `number`).

Contoh Kode / Perintah CLI:

```
or they can be named by using either an interface:

// ----------------------------------------

or a type alias:
```

[typescript_083] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Quick Reference

Sumber: ht

Quick Reference
We have cheat-sheets available for both `type` and `interface`, if you want a quick look at the important every-day syntax at a glance.

[typescript_084] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Property Modifiers

Sumber: ht

Property Modifiers
Each property in an object type can specify a couple of things: the type, whether the property is optional, and whether the property can be written to.

[typescript_085] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Optional Properties

Sumber: ht

Optional Properties
Much of the time, we'll find ourselves dealing with objects that _might_ have a property set.
In those cases, we can mark those properties as _optional_ by adding a question mark (`?`) to the end of their names.
In this example, both `xPos` and `yPos` are considered optional.
We can choose to provide either of them, so every call above to `paintShape` is valid.
All optionality really says is that if the property _is_ set, it better have a specific type.
We can also read from those properties - but when we do under `strictNullChecks`, TypeScript will tell us they're

potentially `undefined`.
In JavaScript, even if the property has never been set, we can still access it - it's just going to give us the value `undefined`.
We can just handle `undefined` specially by checking for it.
Note that this pattern of setting defaults for unspecified values is so common that JavaScript has syntax to support it.
Here we used a destructuring pattern for `paintShape`'s parameter, and provided default values for `xPos` and `yPos`.
Now `xPos` and `yPos` are both definitely present within the body of `paintShape`, but optional for any callers to `paintShape`.
> Note that there is currently no way to place type annotations within destructuring patterns.
> This is because the following syntax already means something different in JavaScript.
>
>
>
> In an object destructuring pattern, `shape: Shape` means "grab the property `shape` and redefine it locally as a variable named `Shape`."
> Likewise `xPos: number` creates a variable named `number` whose value is based on the parameter's `xPos`.

Contoh Kode / Perintah CLI:

```
In this example, both `xPos` and `yPos` are considered optional.
We can choose to provide either of them, so every call above to `paintShape` is valid.
All optionality really says is that if the property _is_ set, it better have a specific type.

We can also read from those properties - but when we do under [`strictNullChecks`](/tsconfig#strictNullChecks), TypeScript will tell us they're potentially `undefined`.

// ----------------------------------------

In JavaScript, even if the property has never been set, we can still access it - it's just going to give us the value `undefined`.
We can just handle `undefined` specially by checking for it.

// ----------------------------------------

Note that this pattern of setting defaults for unspecified values is so common that JavaScript has syntax to support it.
```

[typescript_086] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): `readonly` Properties

Sumber: ht

`readonly` Properties
Properties can also be marked as `readonly` for TypeScript.
While it won't change any behavior at runtime, a property marked as `readonly` can't be written to during type-checking.
Using the `readonly` modifier doesn't necessarily imply that a value is totally immutable - or in other words, that its internal contents can't be changed.
It just means the property itself can't be re-written to.
It's important to manage expectations of what `readonly` implies.
It's useful to signal intent during development time for TypeScript on how an object should be used.
TypeScript doesn't factor in whether properties on two types are `readonly` when checking whether those types are compatible, so `readonly` properties can also change via aliasing.
Using mapping modifiers, you can remove `readonly` attributes.

Contoh Kode / Perintah CLI:

```
Using the `readonly` modifier doesn't necessarily imply that a value is totally immutable - or in other words, that its internal contents can't be changed.
```

```
It just means the property itself can't be re-written to.

// ----------------------------------------

It's important to manage expectations of what `readonly` implies.
It's useful to signal intent during development time for TypeScript on how an object should be used.
TypeScript doesn't factor in whether properties on two types are `readonly` when checking whether those types are compatible, so `readonly` properties can also change via aliasing.
```

[typescript_087] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Index Signatures

Sumber: ht

Index Signatures
Sometimes you don't know all the names of a type's properties ahead of time, but you do know the shape of the values.
In those cases you can use an index signature to describe the types of possible values, for example:
Above, we have a `StringArray` interface which has an index signature.
This index signature states that when a `StringArray` is indexed with a `number`, it will return a `string`.
Only some types are allowed for index signature properties: `string`, `number`, `symbol`, template string patterns, and union types consisting only of these.
It is possible to support multiple types of indexers...
It is possible to support multiple types of indexers. Note that when using both `number` and `string` indexers, the type returned from a numeric indexer must be a subtype of the type returned from the string indexer. This is because when indexing with a number, JavaScript will actually convert that to a string before indexing into an object. That means that indexing with 100 (a number) is the same thing as indexing with "100" (a string), so the two need to be consistent.
While string index signatures are a powerful way to describe the "dictionary" pattern, they also enforce that all properties match their return type.
This is because a string index declares that `obj.property` is also available as `obj["property"]`.
In the following example, `name`'s type does not match the string index's type, and the type checker gives an error:
However, properties of different types are acceptable if the index signature is a union of the property types:
Finally, you can make index signatures `readonly` in order to prevent assignment to their indices:
You can't set `myArray[2]` because the index signature is `readonly`.

Contoh Kode / Perintah CLI:

```
Above, we have a `StringArray` interface which has an index signature.
This index signature states that when a `StringArray` is indexed with a `number`, it will return a `string`.

Only some types are allowed for index signature properties: `string`, `number`, `symbol`, template string patterns, and union types consisting only of these.

<details>
    <summary>It is possible to support multiple types of indexers...</summary>
     <p>It is possible to support multiple types of indexers. Note that when using both `number` and `string` indexers, the type returned from a numeric indexer must be a subtype of the type returned from the string indexer. This is because when indexing with a <code>number</code>, JavaScript will actually convert that to a <code>string</code> before indexing into an object. That means that indexing with <code>100</code> (a <code>number</code>) is the same thing as indexing with <code>"100"</code> (a <code>string</code>), so the two need to be consistent.</p>

// ----------------------------------------

</details>

While string index signatures are a powerful way to describe the "dictionary" pattern, they also enforce that all properties match their return type.
```

```
This is because a string index declares that `obj.property` is also available as `obj["property"]`.
In the following example, `name`'s type does not match the string index's type, and the type checker
gives an error:

// ----------------------------------------

However, properties of different types are acceptable if the index signature is a union of the property
types:
```

[typescript_088] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Excess Property Checks

Sumber: ht

Excess Property Checks
Where and how an object is assigned a type can make a difference in the type system.
One of the key examples of this is in excess property checking, which validates the object more thoroughly when it is created and assigned to an object type during creation.
Notice the given argument to `createSquare` is spelled _`colour`_ instead of `color`.
In plain JavaScript, this sort of thing fails silently.
You could argue that this program is correctly typed, since the `width` properties are compatible, there's no `color` property present, and the extra `colour` property is insignificant.
However, TypeScript takes the stance that there's probably a bug in this code.
Object literals get special treatment and undergo _excess property checking_ when assigning them to other variables, or passing them as arguments.
If an object literal has any properties that the "target type" doesn't have, you'll get an error:
Getting around these checks is actually really simple.
The easiest method is to just use a type assertion:
However, a better approach might be to add a string index signature if you're sure that the object can have some extra properties that are used in some special way.
If `SquareConfig` can have `color` and `width` properties with the above types, but could _also_ have any number of other properties, then we could define it like so:
Here we're saying that `SquareConfig` can have any number of properties, and as long as they aren't `color` or `width`, their types don't matter.
One final way to get around these checks, which might be a bit surprising, is to assign the object to another variable:
Since assigning `squareOptions` won't undergo excess property checks, the compiler won't give you an error:
The above workaround will work as long as you have

Contoh Kode / Perintah CLI:

```
Notice the given argument to `createSquare` is spelled _`colour`_ instead of `color`.
In plain JavaScript, this sort of thing fails silently.

You could argue that this program is correctly typed, since the `width` properties are compatible,
there's no `color` property present, and the extra `colour` property is insignificant.

However, TypeScript takes the stance that there's probably a bug in this code.
Object literals get special treatment and undergo _excess property checking_ when assigning them to other
variables, or passing them as arguments.
If an object literal has any properties that the "target type" doesn't have, you'll get an error:

// ----------------------------------------

Getting around these checks is actually really simple.
The easiest method is to just use a type assertion:

// ----------------------------------------
```

```
However, a better approach might be to add a string index signature if you're sure that the object can have some extra properties that are used in some special way.
If `SquareConfig` can have `color` and `width` properties with the above types, but could _also_ have any number of other properties, then we could define it like so:
```

[typescript_089] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Extending Types

Sumber: ht

Extending Types
It's pretty common to have types that might be more specific versions of other types.
For example, we might have a `BasicAddress` type that describes the fields necessary for sending letters and packages in the U.S.
In some situations that's enough, but addresses often have a unit number associated with them if the building at an address has multiple units.
We can then describe an `AddressWithUnit`.
This does the job, but the downside here is that we had to repeat all the other fields from `BasicAddress` when our changes were purely additive.
Instead, we can extend the original `BasicAddress` type and just add the new fields that are unique to `AddressWithUnit`.
The `extends` keyword on an `interface` allows us to effectively copy members from other named types, and add whatever new members we want.
This can be useful for cutting down the amount of type declaration boilerplate we have to write, and for signaling intent that several different declarations of the same property might be related.
For example, `AddressWithUnit` didn't need to repeat the `street` property, and because `street` originates from `BasicAddress`, a reader will know that those two types are related in some way.
`interface`s can also extend from multiple types.

Contoh Kode / Perintah CLI:

```
In some situations that's enough, but addresses often have a unit number associated with them if the building at an address has multiple units.
We can then describe an `AddressWithUnit`.

<!-- prettier-ignore -->

// ----------------------------------------

This does the job, but the downside here is that we had to repeat all the other fields from `BasicAddress` when our changes were purely additive.
Instead, we can extend the original `BasicAddress` type and just add the new fields that are unique to `AddressWithUnit`.

// ----------------------------------------

The `extends` keyword on an `interface` allows us to effectively copy members from other named types, and add whatever new members we want.
This can be useful for cutting down the amount of type declaration boilerplate we have to write, and for signaling intent that several different declarations of the same property might be related.
For example, `AddressWithUnit` didn't need to repeat the `street` property, and because `street` originates from `BasicAddress`, a reader will know that those two types are related in some way.

`interface`s can also extend from multiple types.
```

[typescript_090] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Intersection Types

Sumber: ht

Intersection Types
`interface`s allowed us to build up new types from other types by extending them.
TypeScript provides another construct called _intersection types_ that is mainly used to combine existing object types.
An intersection type is defined using the `&` operator.
Here, we've intersected `Colorful` and `Circle` to produce a new type that has all the members of `Colorful` _and_ `Circle`.

Contoh Kode / Perintah CLI:

```
Here, we've intersected `Colorful` and `Circle` to produce a new type that has all the members of `Colorful` _and_ `Circle`.
```

[typescript_091] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Interface Extension vs. Intersection

Sumber: ht

Interface Extension vs. Intersection
We just looked at two ways to combine types which are similar, but are actually subtly different.
With interfaces, we could use an `extends` clause to extend from other types, and we were able to do something similar with intersections and name the result with a type alias.
The principal difference between the two is how conflicts are handled, and that difference is typically one of the main reasons why you'd pick one over the other between an interface and a type alias of an intersection type.
If interfaces are defined with the same name, TypeScript will attempt to merge them if the properties are compatible. If the properties are not compatible (i.e., they have the same property name but different types), TypeScript will raise an error.
In the case of intersection types, properties with different types will be merged automatically. When the type is used later, TypeScript will expect the property to satisfy both types simultaneously, which may produce unexpected results.
For example, the following code will throw an error because the properties are incompatible:
In contrast, the following code will compile, but it results in a `never` type:
In this case, Staff would require the name property to be both a string and a number, which results in property being of type `never`.

Contoh Kode / Perintah CLI:

```
interface Person {
  name: string;
}

interface Person {
  name: number;
}
```

[typescript_092] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Generic Object Types

Sumber: ht

Generic Object Types
Let's imagine a `Box` type that can contain any value - `string`s, `number`s, `Giraffe`s, whatever.
Right now, the `contents` property is typed as `any`, which works, but can lead to accidents down the line.
We could instead use `unknown`, but that would mean that in cases where we already know the type of `contents`, we'd

need to do precautionary checks, or use error-prone type assertions.
One type safe approach would be to instead scaffold out different `Box` types for every type of `contents`.
But that means we'll have to create different functions, or overloads of functions, to operate on these types.
That's a lot of boilerplate. Moreover, we might later need to introduce new types and overloads.
This is frustrating, since our box types and overloads are all effectively the same.
Instead, we can make a _generic_ `Box` type which declares a _type parameter_.
You might read this as "A `Box` of `Type` is something whose `contents` have type `Type`".
Later on, when we refer to `Box`, we have to give a _type argument_ in place of `Type`.
Think of `Box` as a template for a real type, where `Type` is a placeholder that will get replaced with some other type.
When TypeScript sees `Box`, it will replace every instance of `Type` in `Box` with `string`, and end up working with something like `{ contents: string }`.
In other words, `Box` and our earlier `StringBox` work identically.
`Box` is reusable in that `Type` can be substituted with anything. That means that when we need a box for a new type, we don't need to declare a new `Box` type at all (though we certainly could if we wanted to).
This also means that we can avoid overloads entirely by instead using generic functions.
It is worth noting that type aliases can also be generic. We could have defin

Contoh Kode / Perintah CLI:

```
Right now, the `contents` property is typed as `any`, which works, but can lead to accidents down the line.

We could instead use `unknown`, but that would mean that in cases where we already know the type of `contents`, we'd need to do precautionary checks, or use error-prone type assertions.

// ----------------------------------------

One type safe approach would be to instead scaffold out different `Box` types for every type of `contents`.

// ----------------------------------------

But that means we'll have to create different functions, or overloads of functions, to operate on these types.
```

[typescript_093] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): The `Array` Type

Sumber: ht

The `Array` Type
Generic object types are often some sort of container type that work independently of the type of elements they contain.
It's ideal for data structures to work this way so that they're re-usable across different data types.
It turns out we've been working with a type just like that throughout this handbook: the `Array` type.
Whenever we write out types like `number[]` or `string[]`, that's really just a shorthand for `Array` and `Array`.
Much like the `Box` type above, `Array` itself is a generic type.
Modern JavaScript also provides other data structures which are generic, like `Map`, `Set`, and `Promise`.
All this really means is that because of how `Map`, `Set`, and `Promise` behave, they can work with any sets of types.

Contoh Kode / Perintah CLI:

```
Much like the `Box` type above, `Array` itself is a generic type.
```

[typescript_094] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): The `ReadonlyArray` Type

Sumber: ht

The `ReadonlyArray` Type
The `ReadonlyArray` is a special type that describes arrays that shouldn't be changed.
Much like the `readonly` modifier for properties, it's mainly a tool we can use for intent.
When we see a function that returns `ReadonlyArray`s, it tells us we're not meant to change the contents at all, and when we see a function that consumes `ReadonlyArray`s, it tells us that we can pass any array into that function without worrying that it will change its contents.
Unlike `Array`, there isn't a `ReadonlyArray` constructor that we can use.
Instead, we can assign regular `Array`s to `ReadonlyArray`s.
Just as TypeScript provides a shorthand syntax for `Array` with `Type[]`, it also provides a shorthand syntax for `ReadonlyArray` with `readonly Type[]`.
One last thing to note is that unlike the `readonly` property modifier, assignability isn't bidirectional between regular `Array`s and `ReadonlyArray`s.

Contoh Kode / Perintah CLI:

```
Much like the `readonly` modifier for properties, it's mainly a tool we can use for intent.
When we see a function that returns `ReadonlyArray`s, it tells us we're not meant to change the contents at all, and when we see a function that consumes `ReadonlyArray`s, it tells us that we can pass any array into that function without worrying that it will change its contents.

Unlike `Array`, there isn't a `ReadonlyArray` constructor that we can use.

// ----------------------------------------

Instead, we can assign regular `Array`s to `ReadonlyArray`s.

// ----------------------------------------

Just as TypeScript provides a shorthand syntax for `Array<Type>` with `Type[]`, it also provides a shorthand syntax for `ReadonlyArray<Type>` with `readonly Type[]`.
```

[typescript_095] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): Tuple Types

Sumber: ht

Tuple Types
A _tuple type_ is another sort of `Array` type that knows exactly how many elements it contains, and exactly which types it contains at specific positions.
Here, `StringNumberPair` is a tuple type of `string` and `number`.
Like `ReadonlyArray`, it has no representation at runtime, but is significant to TypeScript.
To the type system, `StringNumberPair` describes arrays whose `0` index contains a `string` and whose `1` index contains a `number`.
If we try to index past the number of elements, we'll get an error.
We can also destructure tuples using JavaScript's array destructuring.
> Tuple types are useful in heavily convention-based APIs, where each element's meaning is "obvious".
> This gives us flexibility in whatever we want to name our variables when we destructure them.
> In the above example, we were able to name elements `0` and `1` to whatever we wanted.
>
> However, since not every user holds the same view of what's obvious, it may be worth reconsidering whether using objects with descriptive property names may be better for your API.

Other than those length checks, simple tuple types like these are equivalent to types which are versions of `Array`s that declare properties for specific indexes, and that declare `length` with a numeric literal type.
Another thing you may be interested in is that tuples can have optional properties by writing out a question mark (`?` after an element's type).
Optional tuple elements can only come at the end, and also affect the type of `length`.
Tuples can also have rest elements, which have to be an array/tuple type.
- `StringNumberBooleans` describes a tuple whose first two elements are `string` and `number` respectively, but which may have any number of `boolean`s following.
- `StringBooleansNumber` describes a

Contoh Kode / Perintah CLI:

```
Here, `StringNumberPair` is a tuple type of `string` and `number`.
Like `ReadonlyArray`, it has no representation at runtime, but is significant to TypeScript.
To the type system, `StringNumberPair` describes arrays whose `0` index contains a `string` and whose `1`
index contains a `number`.

// ----------------------------------------

If we try to index past the number of elements, we'll get an error.

// ----------------------------------------

We                          can                          also                          [destructure
tuples](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring_assignm
ent#Array_destructuring) using JavaScript's array destructuring.
```

[typescript_096] TypeScript - Object Types & Interfaces

Object Types (Property Modifiers, readonly, Index Signatures, Extending Types): `readonly` Tuple Types

Sumber: ht

`readonly` Tuple Types
One final note about tuple types - tuple types have `readonly` variants, and can be specified by sticking a `readonly` modifier in front of them - just like with array shorthand syntax.
As you might expect, writing to any property of a `readonly` tuple isn't allowed in TypeScript.
Tuples tend to be created and left un-modified in most code, so annotating types as `readonly` tuples when possible is a good default.
This is also important given that array literals with `const` assertions will be inferred with `readonly` tuple types.
Here, `distanceFromOrigin` never modifies its elements, but expects a mutable tuple.
Since `point`'s type was inferred as `readonly [3, 4]`, it won't be compatible with `[number, number]` since that type can't guarantee `point`'s elements won't be mutated.
<!-- ## Other Kinds of Object Members
Most of the declarations in object types:

Contoh Kode / Perintah CLI:

```
As you might expect, writing to any property of a `readonly` tuple isn't allowed in TypeScript.

// ----------------------------------------

Tuples tend to be created and left un-modified in most code, so annotating types as `readonly` tuples
when possible is a good default.
This is also important given that array literals with `const` assertions will be inferred with `readonly`
tuple types.
```

[typescript_097] TypeScript - Generics & Type Manipulation

Generics (Type Variables, Generic Types, Generic Classes, Constraints): Overview

Sumber: ht

A major part of software engineering is building components that not only have well-defined and consistent APIs, but are also reusable.
Components that are capable of working on the data of today as well as the data of tomorrow will give you the most flexible capabilities for building up large software systems.
In languages like C# and Java, one of the main tools in the toolbox for creating reusable components is _generics_, that is, being able to create a component that can work over a variety of types rather than a single one.
This allows users to consume these components and use their own types.

[typescript_098] TypeScript - Generics & Type Manipulation

Generics (Type Variables, Generic Types, Generic Classes, Constraints): Hello World of Generics

Sumber: ht

Hello World of Generics
To start off, let's do the "hello world" of generics: the identity function.
The identity function is a function that will return back whatever is passed in.
You can think of this in a similar way to the `echo` command.
Without generics, we would either have to give the identity function a specific type:
Or, we could describe the identity function using the `any` type:
While using `any` is certainly generic in that it will cause the function to accept any and all types for the type of `arg`, we actually are losing the information about what that type was when the function returns.
If we passed in a number, the only information we have is that any type could be returned.
Instead, we need a way of capturing the type of the argument in such a way that we can also use it to denote what is being returned.
Here, we will use a _type variable_, a special kind of variable that works on types rather than values.
We've now added a type variable `Type` to the identity function.
This `Type` allows us to capture the type the user provides (e.g. `number`), so that we can use that information later.
Here, we use `Type` again as the return type. On inspection, we can now see the same type is used for the argument and the return type.
This allows us to traffic that type information in one side of the function and out the other.
We say that this version of the `identity` function is generic, as it works over a range of types.
Unlike using `any`, it's also just as precise (i.e., it doesn't lose any information) as the first `identity` function that used numbers for the argument and return type.
Once we've written the generic identity function, we can call it in one of two ways.
The first way is to pass all of the arguments, including the type argument, to the fun

Contoh Kode / Perintah CLI:

```
Or, we could describe the identity function using the `any` type:

// ----------------------------------------

While using `any` is certainly generic in that it will cause the function to accept any and all types for the type of `arg`, we actually are losing the information about what that type was when the function returns.
If we passed in a number, the only information we have is that any type could be returned.

Instead, we need a way of capturing the type of the argument in such a way that we can also use it to denote what is being returned.
Here, we will use a _type variable_, a special kind of variable that works on types rather than values.

// ----------------------------------------
```

```
We've now added a type variable `Type` to the identity function.
This `Type` allows us to capture the type the user provides (e.g. `number`), so that we can use that information later.
Here, we use `Type` again as the return type. On inspection, we can now see the same type is used for the argument and the return type.
This allows us to traffic that type information in one side of the function and out the other.

We say that this version of the `identity` function is generic, as it works over a range of types.
Unlike using `any`, it's also just as precise (i.e., it doesn't lose any information) as the first `identity` function that used numbers for the argument and return type.

Once we've written the generic identity function, we can call it in one of two ways.
The first way is to pass all of the arguments, including the type argument, to the function:
```

[typescript_099] TypeScript - Generics & Type Manipulation

Generics (Type Variables, Generic Types, Generic Classes, Constraints): Working with Generic Type Variables

Sumber: ht

Working with Generic Type Variables

When you begin to use generics, you'll notice that when you create generic functions like `identity`, the compiler will enforce that you use any generically typed parameters in the body of the function correctly.

That is, that you actually treat these parameters as if they could be any and all types.

Let's take our `identity` function from earlier:

What if we want to also log the length of the argument `arg` to the console with each call?

We might be tempted to write this:

When we do, the compiler will give us an error that we're using the `.length` member of `arg`, but nowhere have we said that `arg` has this member.

Remember, we said earlier that these type variables stand in for any and all types, so someone using this function could have passed in a `number` instead, which does not have a `.length` member.

Let's say that we've actually intended this function to work on arrays of `Type` rather than `Type` directly. Since we're working with arrays, the `.length` member should be available.

We can describe this just like we would create arrays of other types:

You can read the type of `loggingIdentity` as "the generic function `loggingIdentity` takes a type parameter `Type`, and an argument `arg` which is an array of `Type`s, and returns an array of `Type`s."

If we passed in an array of numbers, we'd get an array of numbers back out, as `Type` would bind to `number`.

This allows us to use our generic type variable `Type` as part of the types we're working with, rather than the whole type, giving us greater flexibility.

We can alternatively write the sample example this way:

You may already be familiar with this style of type from other languages.

In the next section, we'll cover how you can create your own generic types like `Array

Contoh Kode / Perintah CLI:

```
What if we want to also log the length of the argument `arg` to the console with each call?
We might be tempted to write this:

// ----------------------------------------

When we do, the compiler will give us an error that we're using the `.length` member of `arg`, but nowhere have we said that `arg` has this member.
Remember, we said earlier that these type variables stand in for any and all types, so someone using this function could have passed in a `number` instead, which does not have a `.length` member.

Let's say that we've actually intended this function to work on arrays of `Type` rather than `Type` directly. Since we're working with arrays, the `.length` member should be available.
We can describe this just like we would create arrays of other types:
```

```
// ----------------------------------------

You can read the type of `loggingIdentity` as "the generic function `loggingIdentity` takes a type parameter `Type`, and an argument `arg` which is an array of `Type`s, and returns an array of `Type`s."
If we passed in an array of numbers, we'd get an array of numbers back out, as `Type` would bind to `number`.
This allows us to use our generic type variable `Type` as part of the types we're working with, rather than the whole type, giving us greater flexibility.

We can alternatively write the sample example this way:
```

[typescript_100] TypeScript - Generics & Type Manipulation

Generics (Type Variables, Generic Types, Generic Classes, Constraints): Generic Types

Sumber: ht

Generic Types
In previous sections, we created generic identity functions that worked over a range of types.
In this section, we'll explore the type of the functions themselves and how to create generic interfaces.
The type of generic functions is just like those of non-generic functions, with the type parameters listed first, similarly to function declarations:
We could also have used a different name for the generic type parameter in the type, so long as the number of type variables and how the type variables are used line up.
We can also write the generic type as a call signature of an object literal type:
Which leads us to writing our first generic interface.
Let's take the object literal from the previous example and move it to an interface:
In a similar example, we may want to move the generic parameter to be a parameter of the whole interface.
This lets us see what type(s) we're generic over (e.g. `Dictionary` rather than just `Dictionary`).
This makes the type parameter visible to all the other members of the interface.
Notice that our example has changed to be something slightly different.
Instead of describing a generic function, we now have a non-generic function signature that is a part of a generic type.
When we use `GenericIdentityFn`, we now will also need to specify the corresponding type argument (here: `number`), effectively locking in what the underlying call signature will use.
Understanding when to put the type parameter directly on the call signature and when to put it on the interface itself will be helpful in describing what aspects of a type are generic.
In addition to generic interfaces, we can also create generic classes.
Note that it is not possible to create generic enums and namespaces.

Contoh Kode / Perintah CLI:

```
We could also have used a different name for the generic type parameter in the type, so long as the number of type variables and how the type variables are used line up.

// ----------------------------------------

We can also write the generic type as a call signature of an object literal type:

// ----------------------------------------

Which leads us to writing our first generic interface.
Let's take the object literal from the previous example and move it to an interface:
```

[typescript_101] TypeScript - Generics & Type Manipulation

Generics (Type Variables, Generic Types, Generic Classes, Constraints): Generic Classes

Sumber: ht

Generic Classes
A generic class has a similar shape to a generic interface.
Generic classes have a generic type parameter list in angle brackets (`<>`) following the name of the class.
This is a pretty literal use of the `GenericNumber` class, but you may have noticed that nothing is restricting it to only use the `number` type.
We could have instead used `string` or even more complex objects.
Just as with interface, putting the type parameter on the class itself lets us make sure all of the properties of the class are working with the same type.
As we cover in our section on classes, a class has two sides to its type: the static side and the instance side.
Generic classes are only generic over their instance side rather than their static side, so when working with classes, static members can not use the class's type parameter.

Contoh Kode / Perintah CLI:

```
This is a pretty literal use of the `GenericNumber` class, but you may have noticed that nothing is restricting it to only use the `number` type.
We could have instead used `string` or even more complex objects.
```

[typescript_102] TypeScript - Generics & Type Manipulation

Generics (Type Variables, Generic Types, Generic Classes, Constraints): Generic Constraints

Sumber: ht

Generic Constraints
If you remember from an earlier example, you may sometimes want to write a generic function that works on a set of types where you have _some_ knowledge about what capabilities that set of types will have.
In our `loggingIdentity` example, we wanted to be able to access the `.length` property of `arg`, but the compiler could not prove that every type had a `.length` property, so it warns us that we can't make this assumption.
Instead of working with any and all types, we'd like to constrain this function to work with any and all types that *also* have the `.length` property.
As long as the type has this member, we'll allow it, but it's required to have at least this member.
To do so, we must list our requirement as a constraint on what `Type` can be.
To do so, we'll create an interface that describes our constraint.
Here, we'll create an interface that has a single `.length` property and then we'll use this interface and the `extends` keyword to denote our constraint:
Because the generic function is now constrained, it will no longer work over any and all types:
Instead, we need to pass in values whose type has all the required properties:

Contoh Kode / Perintah CLI:

```
Instead of working with any and all types, we'd like to constrain this function to work with any and all types that *also*  have the `.length` property.
As long as the type has this member, we'll allow it, but it's required to have at least this member.
To do so, we must list our requirement as a constraint on what `Type` can be.

To do so, we'll create an interface that describes our constraint.
Here, we'll create an interface that has a single `.length` property and then we'll use this interface and the `extends` keyword to denote our constraint:

// ----------------------------------------

Because the generic function is now constrained, it will no longer work over any and all types:
```

```
// ----------------------------------------

Instead, we need to pass in values whose type has all the required properties:
```

[typescript_103] TypeScript - Generics & Type Manipulation

Generics (Type Variables, Generic Types, Generic Classes, Constraints): Using Type Parameters in Generic Constraints

Sumber: ht

Using Type Parameters in Generic Constraints
You can declare a type parameter that is constrained by another type parameter.
For example, here we'd like to get a property from an object given its name.
We'd like to ensure that we're not accidentally grabbing a property that does not exist on the `obj`, so we'll place a constraint between the two types:

[typescript_104] TypeScript - Generics & Type Manipulation

Generics (Type Variables, Generic Types, Generic Classes, Constraints): Using Class Types in Generics

Sumber: ht

Using Class Types in Generics
When creating factories in TypeScript using generics, it is necessary to refer to class types by their constructor functions. For example,
A more advanced example uses the prototype property to infer and constrain relationships between the constructor function and the instance side of class types.
This pattern is used to power the mixins design pattern.

Contoh Kode / Perintah CLI:

```
A more advanced example uses the prototype property to infer and constrain relationships between the constructor function and the instance side of class types.
```

[typescript_105] TypeScript - Generics & Type Manipulation

Generics (Type Variables, Generic Types, Generic Classes, Constraints): Generic Parameter Defaults

Sumber: ht

Generic Parameter Defaults
By declaring a default for a generic type parameter, you make it optional to specify the corresponding type argument. For example, a function which creates a new `HTMLElement`. Calling the function with no arguments generates a `HTMLDivElement`; calling the function with an element as the first argument generates an element of the argument's type. You can optionally pass a list of children as well. Previously you would have to define the function as:
With generic parameter defaults we can reduce it to:
A generic parameter default follows the following rules:
- A type parameter is deemed optional if it has a default.
- Required type parameters must not follow optional type parameters.
- Default types for a type parameter must satisfy the constraint for the type parameter, if it exists.
- When specifying type arguments, you are only required to specify type arguments for the required type parameters. Unspecified type parameters will resolve to their default types.
- If a default type is specified and inference cannot choose a candidate, the default type is inferred.
- A class or interface declaration that merges with an existing class or interface declaration may introduce a default for an existing type parameter.
- A class or interface declaration that merges with an existing class or interface declaration may introduce a new type parameter as long as it specifies a default.

Contoh Kode / Perintah CLI:

```
With generic parameter defaults we can reduce it to:
```

[typescript_106] TypeScript - Generics & Type Manipulation

Generics (Type Variables, Generic Types, Generic Classes, Constraints): Variance Annotations

Sumber: ht

Variance Annotations

> This is an advanced feature for solving a very specific problem, and should only be used in situations where you've identified a reason to use it

Covariance and contravariance) are type theory terms that describe what the relationship between two generic types is. Here's a brief primer on the concept.

For example, if you have an interface representing an object that can `make` a certain type:

We can use a `Producer` where a `Producer` is expected, because a `Cat` is an `Animal`.

This relationship is called *covariance*: the relationship from `Producer` to `Producer` is the same as the relationship from `T` to `U`.

Conversely, if you have an interface that can `consume` a certain type:

Then we can use a `Consumer` where a `Consumer` is expected, because any function that is capable of accepting an `Animal` must also be capable of accepting a `Cat`.

This relationship is called *contravariance*: the relationship from `Consumer` to `Consumer` is the same as the relationship from `U` to `T`.

Note the reversal of direction as compared to covariance! This is why contravariance "cancels itself out" but covariance doesn't.

In a structural type system like TypeScript's, covariance and contravariance are naturally emergent behaviors that follow from the definition of types.

Even in the absence of generics, we would see covariant (and contravariant) relationships:

TypeScript has a structural type system, so when comparing two types, e.g. to see if a `Producer` can be used where a `Producer` is expected, the usual algorithm would be structurally expand both of those definitions, and compare their structures.

However, variance allows for an extremely useful optimization: if `Producer` is covariant on `T`, then we can simply check `Cat` and `Animal` instead,

Contoh Kode / Perintah CLI:

```
interface Producer<T> {
  make(): T;
}

// ----------------------------------------

interface Consumer<T> {
  consume: (arg: T) => void;
}

// ----------------------------------------

interface AnimalProducer {
  make(): Animal;
}

// A CatProducer can be used anywhere an
// Animal producer is expected
interface CatProducer {
  make(): Cat;
}
```

[typescript_107] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Overview

Sumber: ht

Background Reading:Classes (MDN)
TypeScript offers full support for the `class` keyword introduced in ES2015.
As with other JavaScript language features, TypeScript adds type annotations and other syntax to allow you to express relationships between classes and other types.

[typescript_108] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Class Members

Sumber: ht

Class Members
Here's the most basic class - an empty one:
This class isn't very useful yet, so let's start adding some members.

[typescript_109] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Fields

Sumber: ht

Fields
A field declaration creates a public writeable property on a class:
As with other locations, the type annotation is optional, but will be an implicit `any` if not specified.
Fields can also have _initializers_; these will run automatically when the class is instantiated:
Just like with `const`, `let`, and `var`, the initializer of a class property will be used to infer its type:
`--strictPropertyInitialization`
The `strictPropertyInitialization` setting controls whether class fields need to be initialized in the constructor.
Note that the field needs to be initialized _in the constructor itself_.
TypeScript does not analyze methods you invoke from the constructor to detect initializations, because a derived class might override those methods and fail to initialize the members.
If you intend to definitely initialize a field through means other than the constructor (for example, maybe an external library is filling in part of your class for you), you can use the _definite assignment assertion operator_, `!`:

Contoh Kode / Perintah CLI:

```
As with other locations, the type annotation is optional, but will be an implicit `any` if not specified.

Fields can also have _initializers_; these will run automatically when the class is instantiated:

// ----------------------------------------

Just like with `const`, `let`, and `var`, the initializer of a class property will be used to infer its
type:

// ----------------------------------------

#### `--strictPropertyInitialization`

The [`strictPropertyInitialization`](/tsconfig#strictPropertyInitialization) setting controls whether
class fields need to be initialized in the constructor.
```

[typescript_110] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `readonly`

Sumber: ht

`readonly`
Fields may be prefixed with the `readonly` modifier.
This prevents assignments to the field outside of the constructor.

[typescript_111] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Constructors

Sumber: ht

Constructors
Background Reading:
Constructor (MDN)
Class constructors are very similar to functions.
You can add parameters with type annotations, default values, and overloads:
There are just a few differences between class constructor signatures and function signatures:
- Constructors can't have type parameters - these belong on the outer class declaration, which we'll learn about later
- Constructors can't have return type annotations - the class instance type is always what's returned
Super Calls
Just as in JavaScript, if you have a base class, you'll need to call `super();` in your constructor body before using any `this.` members:
Forgetting to call `super` is an easy mistake to make in JavaScript, but TypeScript will tell you when it's necessary.

Contoh Kode / Perintah CLI:

```
There are just a few differences between class constructor signatures and function signatures:

- Constructors can't have type parameters - these belong on the outer class declaration, which we'll learn about later
- Constructors can't have return type annotations - the class instance type is always what's returned

#### Super Calls

Just as in JavaScript, if you have a base class, you'll need to call `super();` in your constructor body before using any `this.` members:
```

[typescript_112] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Methods

Sumber: ht

Methods
Background Reading:
Method definitions
A function property on a class is called a _method_.
Methods can use all the same type annotations as functions and constructors:
Other than the standard type annotations, TypeScript doesn't add anything else new to methods.
Note that inside a method body, it is still mandatory to access fields and other methods via `this.`.
An unqualified name in a method body will always refer to something in the enclosing scope:

Contoh Kode / Perintah CLI:

```
Other than the standard type annotations, TypeScript doesn't add anything else new to methods.

Note that inside a method body, it is still mandatory to access fields and other methods via `this.`.
An unqualified name in a method body will always refer to something in the enclosing scope:
```

[typescript_113] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Getters / Setters

Sumber: ht

Getters / Setters
Classes can also have _accessors_:
> Note that a field-backed get/set pair with no extra logic is very rarely useful in JavaScript.
> It's fine to expose public fields if you don't need to add additional logic during the get/set operations.
TypeScript has some special inference rules for accessors:
- If `get` exists but no `set`, the property is automatically `readonly`
- If the type of the setter parameter is not specified, it is inferred from the return type of the getter
Since TypeScript 4.3, it is possible to have accessors with different types for getting and setting.

Contoh Kode / Perintah CLI:

```
> Note that a field-backed get/set pair with no extra logic is very rarely useful in JavaScript.
> It's fine to expose public fields if you don't need to add additional logic during the get/set operations.

TypeScript has some special inference rules for accessors:

- If `get` exists but no `set`, the property is automatically `readonly`
- If the type of the setter parameter is not specified, it is inferred from the return type of the getter

Since [TypeScript 4.3](https://devblogs.microsoft.com/typescript/announcing-typescript-4-3/), it is possible to have accessors with different types for getting and setting.
```

[typescript_114] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Index Signatures

Sumber: ht

Index Signatures
Classes can declare index signatures; these work the same as Index Signatures for other object types:
Because the index signature type needs to also capture the types of methods, it's not easy to usefully use these types.
Generally it's better to store indexed data in another place instead of on the class instance itself.

[typescript_115] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Class Heritage

Sumber: ht

Class Heritage
Like other languages with object-oriented features, classes in JavaScript can inherit from base classes.

[typescript_116] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `implements` Clauses

Sumber: ht

`implements` Clauses
You can use an `implements` clause to check that a class satisfies a particular `interface`.
An error will be issued if a class fails to correctly implement it:
Classes may also implement multiple interfaces, e.g. `class C implements A, B {`.
Cautions

It's important to understand that an `implements` clause is only a check that the class can be treated as the interface type.
It doesn't change the type of the class or its methods _at all_.
A common source of error is to assume that an `implements` clause will change the class type - it doesn't!
In this example, we perhaps expected that `s`'s type would be influenced by the `name: string` parameter of `check`.
It is not - `implements` clauses don't change how the class body is checked or its type inferred.
Similarly, implementing an interface with an optional property doesn't create that property:

Contoh Kode / Perintah CLI:

```
Classes may also implement multiple interfaces, e.g. `class C implements A, B {`.

#### Cautions

It's important to understand that an `implements` clause is only a check that the class can be treated as the interface type.
It doesn't change the type of the class or its methods _at all_.
A common source of error is to assume that an `implements` clause will change the class type - it doesn't!

// ----------------------------------------

In this example, we perhaps expected that `s`'s type would be influenced by the `name: string` parameter of `check`.
It is not - `implements` clauses don't change how the class body is checked or its type inferred.

Similarly, implementing an interface with an optional property doesn't create that property:
```

[typescript_117] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `extends` Clauses

Sumber: ht

`extends` Clauses
Background Reading:
extends keyword (MDN)
Classes may `extend` from a base class.
A derived class has all the properties and methods of its base class, and can also define additional members.
Overriding Methods
Background Reading:
super keyword (MDN)
A derived class can also override a base class field or property.
You can use the `super.` syntax to access base class methods.
Note that because JavaScript classes are a simple lookup object, there is no notion of a "super field".
TypeScript enforces that a derived class is always a subtype of its base class.
For example, here's a legal way to override a method:
It's important that a derived class follow its base class contract.
Remember that it's very common (and always legal!) to refer to a derived class instance through a base class reference:
What if `Derived` didn't follow `Base`'s contract?
If we compiled this code despite the error, this sample would then crash:
Type-only Field Declarations
When `target >= ES2022` or `useDefineForClassFields` is `true`, class fields are initialized after the parent class constructor completes, overwriting any value set by the parent class. This can be a problem when you only want to re-declare a more accurate type for an inherited field. To handle these cases, you can write `declare` to indicate to TypeScript that there should be no runtime effect for this field declaration.
Initialization Order

The order that JavaScript classes initialize can be surprising in some cases.
Let's consider this code:
What happened here?
The order of class initialization, as defined by JavaScript, is:
- The base class fields are initialized
- The base class constructor runs
- The derived class fields are initialized
- The derived class constructor runs
This means tha

Contoh Kode / Perintah CLI:

```
#### Overriding Methods

<blockquote class='bg-reading'>
   <p>Background Reading:<br />
       <a  href='https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/super'>super
keyword (MDN)</a><br/>
   </p>
</blockquote>

A derived class can also override a base class field or property.
You can use the `super.` syntax to access base class methods.
Note that because JavaScript classes are a simple lookup object, there is no notion of a "super field".

TypeScript enforces that a derived class is always a subtype of its base class.

For example, here's a legal way to override a method:

// ----------------------------------------

It's important that a derived class follow its base class contract.
Remember that it's very common (and always legal!) to refer to a derived class instance through a base
class reference:

// ----------------------------------------

What if `Derived` didn't follow `Base`'s contract?
```

[typescript_118] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Member Visibility

Sumber: ht

Member Visibility
You can use TypeScript to control whether certain methods or properties are visible to code outside the class.

[typescript_119] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `public`

Sumber: ht

`public`
The default visibility of class members is `public`.
A `public` member can be accessed anywhere:
Because `public` is already the default visibility modifier, you don't ever _need_ to write it on a class member, but might choose to do so for style/readability reasons.

[typescript_120] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `protected`

Sumber: ht

`protected`
`protected` members are only visible to subclasses of the class they're declared in.
Exposure of `protected` members
Derived classes need to follow their base class contracts, but may choose to expose a subtype of base class with more capabilities.
This includes making `protected` members `public`:
Note that `Derived` was already able to freely read and write `m`, so this doesn't meaningfully alter the "security" of this situation.
The main thing to note here is that in the derived class, we need to be careful to repeat the `protected` modifier if this exposure isn't intentional.
Cross-hierarchy `protected` access
TypeScript doesn't allow accessing `protected` members of a sibling class in a class hierarchy:
This is because accessing `x` in `Derived2` should only be legal from `Derived2`'s subclasses, and `Derived1` isn't one of them.
Moreover, if accessing `x` through a `Derived1` reference is illegal (which it certainly should be!), then accessing it through a base class reference should never improve the situation.
See also Why Can't I Access A Protected Member From A Derived Class? which explains more of C#'s reasoning on the same topic.

Contoh Kode / Perintah CLI:

```
#### Exposure of `protected` members

Derived classes need to follow their base class contracts, but may choose to expose a subtype of base class with more capabilities.
This includes making `protected` members `public`:

// ----------------------------------------

Note that `Derived` was already able to freely read and write `m`, so this doesn't meaningfully alter the "security" of this situation.
The main thing to note here is that in the derived class, we need to be careful to repeat the `protected` modifier if this exposure isn't intentional.

#### Cross-hierarchy `protected` access

TypeScript doesn't allow accessing `protected` members of a sibling class in a class hierarchy:
```

[typescript_121] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `private`

Sumber: ht

`private`
`private` is like `protected`, but doesn't allow access to the member even from subclasses:
Because `private` members aren't visible to derived classes, a derived class can't increase their visibility:
Cross-instance `private` access
Different OOP languages disagree about whether different instances of the same class may access each others' `private` members.
While languages like Java, C#, C++, Swift, and PHP allow this, Ruby does not.
TypeScript does allow cross-instance `private` access:
Caveats
Like other aspects of TypeScript's type system, `private` and `protected` are only enforced during type checking.
This means that JavaScript runtime constructs like `in` or simple property lookup can still access a `private` or

`protected` member:
`private` also allows access using bracket notation during type checking. This makes `private`-declared fields potentially easier to access for things like unit tests, with the drawback that these fields are _soft private_ and don't strictly enforce privacy.
Unlike TypeScripts's `private`, JavaScript's private fields (`#`) remain private after compilation and do not provide the previously mentioned escape hatches like bracket notation access, making them _hard private_.
When compiling to ES2021 or less, TypeScript will use WeakMaps in place of `#`.
If you need to protect values in your class from malicious actors, you should use mechanisms that offer hard runtime privacy, such as closures, WeakMaps, or private fields. Note that these added privacy checks during runtime could affect performance.

Contoh Kode / Perintah CLI:

```
Because `private` members aren't visible to derived classes, a derived class can't increase their visibility:

// ----------------------------------------

#### Cross-instance `private` access

Different OOP languages disagree about whether different instances of the same class may access each others' `private` members.
While languages like Java, C#, C++, Swift, and PHP allow this, Ruby does not.

TypeScript does allow cross-instance `private` access:

// ----------------------------------------

#### Caveats

Like other aspects of TypeScript's type system, `private` and `protected` [are only enforced during type checking](https://www.typescriptlang.org/play?removeComments=true&target=99&ts=4.3.4#code/PTAEGMBsEMGddAEQPYHNQBMCmVoCcsEAHPASwDdoAXLUAM1K0gwQFdZSA7dAKWkoDK4MkSoByBAGJQJLAwAeAWABQIUH0HDSoiTLKUaoUggAW+DHorUsAOlABJcQlhUy4KpACeoLJzrI8cCwMGxU1ABVPIiwhESpMZEJQTmR4lxFQaQxWMm4IZABbIlIYKlJkTlDlXHgkNFAAbxVQTIAjfABrAEEC5FZOeIBeUAAGAG5mmSw8WAroSFIqb2GAIjMiIk8VieVJ8Ar01ncAgAoASkaAXxVr3dUwGoQAYWpMHBgCYn1rekZmNg4eUi0Vi2icoBWJCsNBWoA6WE8AHcAiEwmBgTEtDovtDaMZQLM6PEoQZbA5wSk0q5SO4vD4-AEghZoJwLGYEIRwNBoqAzFRwCZCFUIlFMXECdSiAhId8YZgclx0PsiiVqOVOAAaUAFLAsxWgKiC35MFigfC0FKgSAVVDTSyk+W5dB4fplHVVR6gF7xJrKFotEk-HXIRE9PoDUDDcaTAPTWaceaLZYQlmoPBbHYx-KcQ7HPDnK43FQqfY5+IMDDISPJLCIuqoc47UsuUCofAME3Vzi1r3URvF5QV5A2STtPDdXqunZDgDaYlHnTDrrEAF0dm28B3mDZg6HJwN1+2-hg57ulwNV2NQGoZbjYfNrYiENBwEFaojFiZQK08C-4fFKTVCozWfTgfFgLkeT5AUqiAA).

This means that JavaScript runtime constructs like `in` or simple property lookup can still access a `private` or `protected` member:
```

[typescript_122] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Static Members

Sumber: ht

Static Members
Background Reading:
Static Members (MDN)
Classes may have `static` members.
These members aren't associated with a particular instance of the class.
They can be accessed through the class constructor object itself:
Static members can also use the same `public`, `protected`, and `private` visibility modifiers:
Static members are also inherited:

Contoh Kode / Perintah CLI:

```
Static members can also use the same `public`, `protected`, and `private` visibility modifiers:
```

```
// ----------------------------------------

Static members are also inherited:
```

[typescript_123] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Special Static Names

Sumber: ht

Special Static Names
It's generally not safe/possible to overwrite properties from the `Function` prototype.
Because classes are themselves functions that can be invoked with `new`, certain `static` names can't be used.
Function properties like `name`, `length`, and `call` aren't valid to define as `static` members:

[typescript_124] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Why No Static Classes?

Sumber: ht

Why No Static Classes?
TypeScript (and JavaScript) don't have a construct called `static class` the same way as, for example, C# does.
Those constructs _only_ exist because those languages force all data and functions to be inside a class; because that restriction doesn't exist in TypeScript, there's no need for them.
A class with only a single instance is typically just represented as a normal _object_ in JavaScript/TypeScript.
For example, we don't need a "static class" syntax in TypeScript because a regular object (or even top-level function) will do the job just as well:

[typescript_125] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `static` Blocks in Classes

Sumber: ht

`static` Blocks in Classes
Static blocks allow you to write a sequence of statements with their own scope that can access private fields within the containing class. This means that we can write initialization code with all the capabilities of writing statements, no leakage of variables, and full access to our class's internals.

[typescript_126] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Generic Classes

Sumber: ht

Generic Classes
Classes, much like interfaces, can be generic.
When a generic class is instantiated with `new`, its type parameters are inferred the same way as in a function call:
Classes can use generic constraints and defaults the same way as interfaces.

[typescript_127] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Type Parameters in Static Members

Sumber: ht

Type Parameters in Static Members
This code isn't legal, and it may not be obvious why:
Remember that types are always fully erased!
At runtime, there's only _one_ `Box.defaultValue` property slot.
This means that setting `Box.defaultValue` (if that were possible) would _also_ change `Box.defaultValue` - not good.
The `static` members of a generic class can never refer to the class's type parameters.

[typescript_128] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `this` at Runtime in Classes

Sumber: ht

`this` at Runtime in Classes
Background Reading:
this keyword (MDN)
It's important to remember that TypeScript doesn't change the runtime behavior of JavaScript, and that JavaScript is somewhat famous for having some peculiar runtime behaviors.
JavaScript's handling of `this` is indeed unusual:
Long story short, by default, the value of `this` inside a function depends on _how the function was called_.
In this example, because the function was called through the `obj` reference, its value of `this` was `obj` rather than the class instance.
This is rarely what you want to happen!
TypeScript provides some ways to mitigate or prevent this kind of error.

[typescript_129] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Arrow Functions

Sumber: ht

Arrow Functions
Background Reading:
Arrow functions (MDN)
If you have a function that will often be called in a way that loses its `this` context, it can make sense to use an arrow function property instead of a method definition:
This has some trade-offs:
- The `this` value is guaranteed to be correct at runtime, even for code not checked with TypeScript
- This will use more memory, because each class instance will have its own copy of each function defined this way
- You can't use `super.getName` in a derived class, because there's no entry in the prototype chain to fetch the base class method from

[typescript_130] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `this` parameters

Sumber: ht

`this` parameters
In a method or function definition, an initial parameter named `this` has special meaning in TypeScript.
These parameters are erased during compilation:
TypeScript checks that calling a function with a `this` parameter is done so with a correct context.
Instead of using an arrow function, we can add a `this` parameter to method definitions to statically enforce that the method is called correctly:
This method makes the opposite trade-offs of the arrow function approach:
- JavaScript callers might still use the class method incorrectly without realizing it
- Only one function per class definition gets allocated, rather than one per class instance
- Base method definitions can still be called via `super`.

Contoh Kode / Perintah CLI:

```
TypeScript checks that calling a function with a `this` parameter is done so with a correct context.
Instead of using an arrow function, we can add a `this` parameter to method definitions to statically
enforce that the method is called correctly:
```

[typescript_131] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `this` Types

Sumber: ht

`this` Types
In classes, a special type called `this` refers _dynamically_ to the type of the current class.
Let's see how this is useful:
Here, TypeScript inferred the return type of `set` to be `this`, rather than `Box`.
Now let's make a subclass of `Box`:
You can also use `this` in a parameter type annotation:
This is different from writing `other: Box` -- if you have a derived class, its `sameAs` method will now only accept other instances of that same derived class:

Contoh Kode / Perintah CLI:

```
Here, TypeScript inferred the return type of `set` to be `this`, rather than `Box`.
Now let's make a subclass of `Box`:

// ----------------------------------------

You can also use `this` in a parameter type annotation:

// ----------------------------------------

This is different from writing `other: Box` -- if you have a derived class, its `sameAs` method will now
only accept other instances of that same derived class:
```

[typescript_132] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `this`-based type guards

Sumber: ht

`this`-based type guards
You can use `this is Type` in the return position for methods in classes and interfaces.
When mixed with a type narrowing (e.g. `if` statements) the type of the target object would be narrowed to the specified `Type`.

A common use-case for a this-based type guard is to allow for lazy validation of a particular field. For example, this case removes an `undefined` from the value held inside box when `hasValue` has been verified to be true:

Contoh Kode / Perintah CLI:

```
A common use-case for a this-based type guard is to allow for lazy validation of a particular field. For example, this case removes an `undefined` from the value held inside box when `hasValue` has been verified to be true:
```

[typescript_133] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Parameter Properties

Sumber: ht

Parameter Properties
TypeScript offers special syntax for turning a constructor parameter into a class property with the same name and value.
These are called _parameter properties_ and are created by prefixing a constructor argument with one of the visibility modifiers `public`, `private`, `protected`, or `readonly`.
The resulting field gets those modifier(s):

[typescript_134] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Class Expressions

Sumber: ht

Class Expressions
Background Reading:
Class expressions (MDN)
Class expressions are very similar to class declarations.
The only real difference is that class expressions don't need a name, though we can refer to them via whatever identifier they ended up bound to:

[typescript_135] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Constructor Signatures

Sumber: ht

Constructor Signatures
JavaScript classes are instantiated with the `new` operator. Given the type of a class itself, the InstanceType utility type models this operation.

[typescript_136] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): `abstract` Classes and Members

Sumber: ht

`abstract` Classes and Members
Classes, methods, and fields in TypeScript may be _abstract_.
An _abstract method_ or _abstract field_ is one that hasn't had an implementation provided.
These members must exist inside an _abstract class_, which cannot be directly instantiated.
The role of abstract classes is to serve as a base class for subclasses which do implement all the abstract members.
When a class doesn't have any abstract members, it is said to be _concrete_.
Let's look at an example:
We can't instantiate `Base` with `new` because it's abstract.

Instead, we need to make a derived class and implement the abstract members:
Notice that if we forget to implement the base class's abstract members, we'll get an error:

Contoh Kode / Perintah CLI:

```
We can't instantiate `Base` with `new` because it's abstract.
Instead, we need to make a derived class and implement the abstract members:

// ----------------------------------------

Notice that if we forget to implement the base class's abstract members, we'll get an error:
```

[typescript_137] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Abstract Construct Signatures

Sumber: ht

Abstract Construct Signatures
Sometimes you want to accept some class constructor function that produces an instance of a class which derives from some abstract class.
For example, you might want to write this code:
TypeScript is correctly telling you that you're trying to instantiate an abstract class.
After all, given the definition of `greet`, it's perfectly legal to write this code, which would end up constructing an abstract class:
Instead, you want to write a function that accepts something with a construct signature:
Now TypeScript correctly tells you about which class constructor functions can be invoked - `Derived` can because it's concrete, but `Base` cannot.

Contoh Kode / Perintah CLI:

```
TypeScript is correctly telling you that you're trying to instantiate an abstract class.
After all, given the definition of `greet`, it's perfectly legal to write this code, which would end up
constructing an abstract class:

// ----------------------------------------

Instead, you want to write a function that accepts something with a construct signature:
```

[typescript_138] TypeScript - Classes & Modern OOP

Classes (Members, Getters/Setters, Implements, Extends, Visibility, static): Relationships Between Classes

Sumber: ht

Relationships Between Classes
In most cases, classes in TypeScript are compared structurally, the same as other types.
For example, these two classes can be used in place of each other because they're identical:
Similarly, subtype relationships between classes exist even if there's no explicit inheritance:
This sounds straightforward, but there are a few cases that seem stranger than others.
Empty classes have no members.
In a structural type system, a type with no members is generally a supertype of anything else.
So if you write an empty class (don't!), anything can be used in place of it:

Contoh Kode / Perintah CLI:

```
Similarly, subtype relationships between classes exist even if there's no explicit inheritance:

// ----------------------------------------

This sounds straightforward, but there are a few cases that seem stranger than others.
```

```
Empty classes have no members.
In a structural type system, a type with no members is generally a supertype of anything else.
So if you write an empty class (don't!), anything can be used in place of it:
```

[typescript_139] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): Overview

Sumber: ht

JavaScript has a long history of different ways to handle modularizing code.
Having been around since 2012, TypeScript has implemented support for a lot of these formats, but over time the community and the JavaScript specification has converged on a format called ES Modules (or ES6 modules). You might know it as the `import`/`export` syntax.
ES Modules was added to the JavaScript spec in 2015, and by 2020 had broad support in most web browsers and JavaScript runtimes.
For focus, the handbook will cover both ES Modules and its popular pre-cursor CommonJS `module.exports =` syntax, and you can find information about the other module patterns in the reference section under Modules.

[typescript_140] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): How JavaScript Modules are Defined

Sumber: ht

How JavaScript Modules are Defined
In TypeScript, just as in ECMAScript 2015, any file containing a top-level `import` or `export` is considered a module.
Conversely, a file without any top-level import or export declarations is treated as a script whose contents are available in the global scope (and therefore to modules as well).
Modules are executed within their own scope, not in the global scope.
This means that variables, functions, classes, etc. declared in a module are not visible outside the module unless they are explicitly exported using one of the export forms.
Conversely, to consume a variable, function, class, interface, etc. exported from a different module, it has to be imported using one of the import forms.

[typescript_141] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): Non-modules

Sumber: ht

Non-modules
Before we start, it's important to understand what TypeScript considers a module.
The JavaScript specification declares that any JavaScript files without an `import` declaration, `export`, or top-level `await` should be considered a script and not a module.
Inside a script file variables and types are declared to be in the shared global scope, and it's assumed that you'll either use the `outFile` compiler option to join multiple input files into one output file, or use multiple `` tags in your HTML to load these files (in the correct order!).
If you have a file that doesn't currently have any `import`s or `export`s, but you want to be treated as a module, add the line:
which will change the file to be a module exporting nothing. This syntax works regardless of your module target.

[typescript_142] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): Modules in TypeScript

Sumber: ht

Modules in TypeScript
Additional Reading:
Impatient JS (Modules)
MDN: JavaScript Modules
There are three main things to consider when writing module-based code in TypeScript:
- **Syntax**: What syntax do I want to use to import and export things?
- **Module Resolution**: What is the relationship between module names (or paths) and files on disk?
- **Module Output Target**: What should my emitted JavaScript module look like?

[typescript_143] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): ES Module Syntax

Sumber: ht

ES Module Syntax
A file can declare a main export via `export default`:
This is then imported via:
In addition to the default export, you can have more than one export of variables and functions via the `export` by omitting `default`:
These can be used in another file via the `import` syntax:

Contoh Kode / Perintah CLI:

```
This is then imported via:

// ----------------------------------------

In addition to the default export, you can have more than one export of variables and functions via the
`export` by omitting `default`:

// ----------------------------------------

These can be used in another file via the `import` syntax:
```

[typescript_144] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): Additional Import Syntax

Sumber: ht

Additional Import Syntax
An import can be renamed using a format like `import {old as new}`:
You can mix and match the above syntax into a single `import`:
You can take all of the exported objects and put them into a single namespace using `* as name`:
You can import a file and _not_ include any variables into your current module via `import "./file"`:
In this case, the `import` does nothing. However, all of the code in `maths.ts` was evaluated, which could trigger side-effects which affect other objects.
TypeScript Specific ES Module Syntax
Types can be exported and imported using the same syntax as JavaScript values:
TypeScript has extended the `import` syntax with two concepts for declaring an import of a type:
`import type`
Which is an import statement which can _only_ import types:
Inline `type` imports

TypeScript 4.5 also allows for individual imports to be prefixed with `type` to indicate that the imported reference is a type:

Together these allow a non-TypeScript transpiler like Babel, swc or esbuild to know what imports can be safely removed.

ES Module Syntax with CommonJS Behavior

TypeScript has ES Module syntax which _directly_ correlates to a CommonJS and AMD `require`. Imports using ES Module are _for most cases_ the same as the `require` from those environments, but this syntax ensures you have a 1 to 1 match in your TypeScript file with the CommonJS output:

You can learn more about this syntax in the modules reference page.

Contoh Kode / Perintah CLI:

```
You can mix and match the above syntax into a single `import`:

// ----------------------------------------

You can take all of the exported objects and put them into a single namespace using `* as name`:

// ----------------------------------------

You can import a file and _not_ include any variables into your current module via `import "./file"`:
```

[typescript_145] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): CommonJS Syntax

Sumber: ht

CommonJS Syntax

CommonJS is the format which most modules on npm are delivered in. Even if you are writing using the ES Modules syntax above, having a brief understanding of how CommonJS syntax works will help you debug easier.

[typescript_146] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): Exporting

Sumber: ht

Exporting

Identifiers are exported via setting the `exports` property on a global called `module`.

Then these files can be imported via a `require` statement:

Or you can simplify a bit using the destructuring feature in JavaScript:

Contoh Kode / Perintah CLI:

```
Then these files can be imported via a `require` statement:

// ----------------------------------------

Or you can simplify a bit using the destructuring feature in JavaScript:
```

[typescript_147] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): CommonJS and ES Modules interop

Sumber: ht

CommonJS and ES Modules interop

There is a mis-match in features between CommonJS and ES Modules regarding the distinction between a default import and a module namespace object import. TypeScript has a compiler flag to reduce the friction between the two different sets of constraints with `esModuleInterop`.

[typescript_148] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): TypeScript's Module Resolution Options

Sumber: ht

TypeScript's Module Resolution Options
Module resolution is the process of taking a string from the `import` or `require` statement, and determining what file that string refers to.
TypeScript includes two resolution strategies: Classic and Node. Classic, the default when the compiler option `module` is not `commonjs`, is included for backwards compatibility.
The Node strategy replicates how Node.js works in CommonJS mode, with additional checks for `.ts` and `.d.ts`.
There are many TSConfig flags which influence the module strategy within TypeScript: `moduleResolution`, `baseUrl`, `paths`, `rootDirs`.
For the full details on how these strategies work, you can consult the Module Resolution reference page.

[typescript_149] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): TypeScript's Module Output Options

Sumber: ht

TypeScript's Module Output Options
There are two options which affect the emitted JavaScript output:
- `target` which determines which JS features are downleveled (converted to run in older JavaScript runtimes) and which are left intact
- `module` which determines what code is used for modules to interact with each other
Which `target` you use is determined by the features available in the JavaScript runtime you expect to run the TypeScript code in. That could be: the oldest web browser you support, the lowest version of Node.js you expect to run on or could come from unique constraints from your runtime - like Electron for example.
All communication between modules happens via a module loader, the compiler option `module` determines which one is used.
At runtime the module loader is responsible for locating and executing all dependencies of a module before executing it.
For example, here is a TypeScript file using ES Modules syntax, showcasing a few different options for `module`:

[typescript_150] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): `UMD`

Sumber: ht

`UMD`
> Note that ES2020 is effectively the same as the original `index.ts`.
You can see all of the available options and what their emitted JavaScript code looks like in the TSConfig Reference for `module`.

[typescript_151] TypeScript - Modern Modules & ESM

Modules (ESM, CommonJS, TypeScript Specific Syntax, Resolution): TypeScript namespaces

Sumber: ht

TypeScript namespaces
TypeScript has its own module format called `namespaces` which pre-dates the ES Modules standard. This syntax has a lot of useful features for creating complex definition files, and still sees active use in DefinitelyTyped. While not deprecated, the majority of the features in namespaces exist in ES Modules and we recommend you use that to align with JavaScript's direction. You can learn more about namespaces in the namespaces reference page.

[typescript_152] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: `using` Declarations and Explicit Resource Management

Sumber: ht

`using` Declarations and Explicit Resource Management
TypeScript 5.2 adds support for the upcoming Explicit Resource Management feature in ECMAScript.
Let's explore some of the motivations and understand what the feature brings us.
It's common to need to do some sort of "clean-up" after creating an object.
For example, you might need to close network connections, delete temporary files, or just free up some memory.
Let's imagine a function that creates a temporary file, reads and writes to it for various operations, and then closes and deletes it.
This is fine, but what happens if we need to perform an early exit?
We're starting to see some duplication of clean-up which can be easy to forget.
We're also not guaranteed to close and delete the file if an error gets thrown.
This could be solved by wrapping this all in a `try`/`finally` block.
While this is more robust, it's added quite a bit of "noise" to our code.
There are also other foot-guns we can run into if we start adding more clean-up logic to our `finally` block -- for example, exceptions preventing other resources from being disposed.
This is what the explicit resource management proposal aims to solve.
The key idea of the proposal is to support resource disposal -- this clean-up work we're trying to deal with -- as a first class idea in JavaScript.
This starts by adding a new built-in `symbol` called `Symbol.dispose`, and we can create objects with methods named by `Symbol.dispose`.
For convenience, TypeScript defines a new global type called `Disposable` which describes these.
Later on we can call those methods.
Moving the clean-up logic to `TempFile` itself doesn't buy us much;
we've basically just moved all the clean-up work from the `finally` block into a method, and that's always been possible.
But having

Contoh Kode / Perintah CLI:

```
import * as fs from "fs";

export function doSomeWork() {
    const path = ".some_temp_file";
    const file = fs.openSync(path, "w+");

    // use file...

    // Close the file and delete it.
    fs.closeSync(file);
    fs.unlinkSync(path);
}

// ---------------------------------------

export function doSomeWork() {
    const path = ".some_temp_file";
    const file = fs.openSync(path, "w+");

    // use file...
    if (someCondition()) {
        // do some more work...

        // Close the file and delete it.
        fs.closeSync(file);
        fs.unlinkSync(path);
```

```
        return;
    }

    // Close the file and delete it.
    fs.closeSync(file);
    fs.unlinkSync(path);
}

// -------------------------------------

export function doSomeWork() {
    const path = ".some_temp_file";
    const file = fs.openSync(path, "w+");

    try {
        // use file...

        if (someCondition()) {
            // do some more work...
            return;
        }
    }
    finally {
        // Close the file and delete it.
        fs.closeSync(file);
        fs.unlinkSync(path);
    }
}
```

[typescript_153] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: Decorator Metadata

Sumber: ht

Decorator Metadata
TypeScript 5.2 implements an upcoming ECMAScript feature called decorator metadata.
The key idea of this feature is to make it easy for decorators to create and consume metadata on any class they're used on or within.
Whenever decorator functions are used, they now have access to a new `metadata` property on their context object.
The `metadata` property just holds a simple object.
Since JavaScript lets us add properties arbitrarily, it can be used as a dictionary that is updated by each decorator.
Alternatively, since every `metadata` object will be identical for each decorated portion of a class, it can be used as a key into a `Map`.
After all decorators on or in a class get run, that object can be accessed on the class via `Symbol.metadata`.
This can be useful in a number of different scenarios.
Metadata could possibly be attached for lots of uses like debugging, serialization, or performing dependency injection with decorators.
Since metadata objects are created per decorated class, frameworks can either privately use them as keys into a `Map` or `WeakMap`, or tack properties on as necessary.
For example, let's say we wanted to use decorators to keep track of which properties and accessors are serializable when using `JSON.stringify` like so:
Here, the intent is that only `age` and `fullName` should be serialized because they are marked with the `@serialize` decorator.
We define a `toJSON` method for this purpose, but it just calls out to `jsonify` which uses the metadata that `@serialize` created.
Here's an example of how the module `./serialize.ts` might be defined:

This module has a local `symbol` called `serializables` to store and retrieve the names of properties marked `@serializable`.
It stores a list of these property names on the metada

Contoh Kode / Perintah CLI:

```
interface Context {
    name: string;
    metadata: Record<PropertyKey, unknown>;
}

function setMetadata(_target: any, context: Context) {
    context.metadata[context.name] = true;
}

class SomeClass {
    @setMetadata
    foo = 123;

    @setMetadata
    accessor bar = "hello!";

    @setMetadata
    baz() { }
}

const ourMetadata = SomeClass[Symbol.metadata];

console.log(JSON.stringify(ourMetadata));
// { "bar": true, "baz": true, "foo": true }

// ----------------------------------------

import { serialize, jsonify } from "./serializer";

class Person {
    firstName: string;
    lastName: string;

    @serialize
    age: number

    @serialize
    get fullName() {
        return `${this.firstName} ${this.lastName}`;
    }

    toJSON() {
        return jsonify(this)
    }

    constructor(firstName: string, lastName: string, age: number) {
        // ...
    }
}

// ----------------------------------------

const serializables = Symbol();

type Context =
    | ClassAccessorDecoratorContext
    | ClassGetterDecoratorContext
```

```
    | ClassFieldDecoratorContext
    ;

export function serialize(_target: any, context: Context): void {
    if (context.static || context.private) {
        throw new Error("Can only serialize public instance members.")
    }
    if (typeof context.name === "symbol") {
        throw new Error("Cannot serialize symbol-named properties.");
    }

    const propNames =
        (context.metadata[serializables] as string[] | undefined) ??= [];
    propNames.push(context.name);
}

export function jsonify(instance: object): string {
    const metadata = instance.constructor[Symbol.metadata];
    const propNames = metadata?.[serializables] as string[] | undefined;
    if (!propNames) {
        throw new Error("No members marked with @serialize.");
    }

    const pairStrings = propNames.map(key => {
        const strKey = JSON.stringify(key);
        const strValue = JSON.stringify((instance as any)[key]);
        return `${strKey}: ${strValue}`;
    });

    return `{ ${pairStrings.join(", ")} }`;
}
```

[typescript_154] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: Named and Anonymous Tuple Elements

Sumber: ht

Named and Anonymous Tuple Elements

Tuple types have supported optional labels or names for each element.

These labels don't change what you're allowed to do with them -- they're solely to help with readability and tooling.

However, TypeScript previously had a rule that tuples could not mix and match between labeled and unlabeled elements.

In other words, either no element could have a label in a tuple, or all elements needed one.

This could be annoying for rest elements where we'd be forced to just add a label like `rest` or `tail`.

It also meant that this restriction had to be enforced internally in the type system, meaning TypeScript would lose labels.

In TypeScript 5.2, the all-or-nothing restriction on tuple labels has been lifted.

The language can now also preserve labels when spreading into an unlabeled tuple.

We'd like to extend our thanks to Josh Goldberg and Mateusz Burzy?ski who collaborated to lift this restriction.

Contoh Kode / Perintah CLI:

```
type Pair<T> = [first: T, second: T];

// ----------------------------------------

// ? fine - no labels
type Pair1<T> = [T, T];

// ? fine - all fully labeled
type Pair2<T> = [first: T, second: T];
```

```
// [X] previously an error
type Pair3<T> = [first: T, T];
//                         ~
// Tuple members must all have names
// or all not have names.

// ----------------------------------------

// [X] previously an error
type TwoOrMore_A<T> = [first: T, second: T, ...T[]];
//                                          ~~~~~~
// Tuple members must all have names
// or all not have names.

// ?
type TwoOrMore_B<T> = [first: T, second: T, rest: ...T[]];
```

[typescript_155] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: Easier Method Usage for Unions of Arrays

Sumber: ht

Easier Method Usage for Unions of Arrays
In previous versions of TypeScript, calling a method on a union of arrays could end in pain.
In this example, TypeScript would try to see if each version of `filter` is compatible across `string[]` and `number[]`.
Without a coherent strategy, TypeScript threw its hands in the air and said "I can't make it work".
In TypeScript 5.2, before giving up in these cases, unions of arrays are treated as a special case.
A new array type is constructed out of each member's element type, and then the method is invoked on that.
Taking the above example, `string[] | number[]` is transformed into `(string | number)[]` (or `Array`), and `filter` is invoked on that type.
There is a slight caveat which is that `filter` will produce an `Array` instead of a `string[] | number[]`;
but for a freshly produced value there is less risk of something "going wrong".
This means lots of methods like `filter`, `find`, `some`, `every`, and `reduce` should all be invokable on unions of arrays in cases where they were not previously.
You can read up more details on the implementing pull request.

Contoh Kode / Perintah CLI:

```
declare let array: string[] | number[];

array.filter(x => !!x);
//    ~~~~~~ error!
// This expression is not callable.
//   Each member of the union type '...' has signatures,
//   but none of those signatures are compatible
//   with each other.
```

[typescript_156] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: Type-Only Import Paths with TypeScript Implementation File Extensions

Sumber: ht

Type-Only Import Paths with TypeScript Implementation File Extensions
TypeScript now allows both declaration *and* implementation file extensions to be included in type-only import paths, regardless of whether `allowImportingTsExtensions` is enabled.
This means that you can now write `import type` statements that use `.ts`, `.mts`, `.cts`, and `.tsx` file extensions.
It also means that `import()` types, which can be used in both TypeScript and JavaScript with JSDoc, can use those file

extensions.
For more information, see the change here.

Contoh Kode / Perintah CLI:

```
import type { JustAType } from "./justTypes.ts";

export function f(param: JustAType) {
    // ...
}

// ---------------------------------------

/**
 * @param {import("./justTypes.ts").JustAType} param
 */
export function f(param) {
    // ...
}
```

[typescript_157] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: Comma Completions for Object Members

Sumber: ht

Comma Completions for Object Members
It can be easy to forget to add a comma when adding a new property to an object.
Previously, if you forgot a comma and requested auto-completion, TypeScript would confusingly give poor unrelated completion results.
TypeScript 5.2 now gracefully provides object member completions when you're missing a comma.
But to just skip past hitting you with a syntax error, it will *also* auto-insert the missing comma.
!Properties in an object literal are completed despite missing a comma after a prior property. When the property name is completed, the missing comma is automatically inserted.
For more information, see the implementation here.

[typescript_158] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: Inline Variable Refactoring

Sumber: ht

Inline Variable Refactoring
TypeScript 5.2 now has a refactoring to inline the contents of a variable to all usage sites.
!A variable called 'path' initialized to a string, having both of its usages replaced.
Using the "inline variable" refactoring will eliminate the variable and replace all the variable's usages with its initializer.
Note that this may cause that initializer's side-effects to run at a different time, and as many times as the variable has been used.
For more details, see the implementing pull request.

[typescript_159] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: Optimized Checks for Ongoing Type Compatibility

Sumber: ht

Optimized Checks for Ongoing Type Compatibility
Because TypeScript is a structural type system, types occasionally need to be compared in a member-wise fashion; however, recursive types add some issues here.
For example:
When checking whether the type `A` is compatible with the type `B`, TypeScript will end up checking whether the types of `value` in `A` and `B` are respectively compatible.
At this point, the type system needs to stop checking any further and proceed to check other members.
To do this, the type system has to track when any two types are already being related.
Previously TypeScript already kept a stack of type pairs, and iterated through that to determine whether those types are being related.
When this stack is shallow that's not a problem; but when the stack isn't shallow, that, uh, is a problem.
In TypeScript 5.3, a simple `Set` helps track this information.
This reduced the time spent on a reported test case that used the drizzle library by over 33%!
Read more on the change here.

Contoh Kode / Perintah CLI:

```
interface A {
    value: A;
    other: string;
}

interface B {
    value: B;
    other: number;
}

// ----------------------------------------

Benchmark 1: old
  Time (mean ± ?):      3.115 s ±  0.067 s    [User: 4.403 s, System: 0.124 s]
  Range (min ... max):    3.018 s ...  3.196 s    10 runs

Benchmark 2: new
  Time (mean ± ?):      2.072 s ±  0.050 s    [User: 3.355 s, System: 0.135 s]
  Range (min ... max):    1.985 s ...  2.150 s    10 runs

Summary
  'new' ran
    1.50 ± 0.05 times faster than 'old'
```

[typescript_160] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: Breaking Changes and Correctness Fixes

Sumber: ht

Breaking Changes and Correctness Fixes
TypeScript strives not to unnecessarily introduce breaks;
however, occasionally we must make corrections and improvements so that code can be better-analyzed.

[typescript_161] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: `lib.d.ts` Changes

Sumber: ht

`lib.d.ts` Changes
Types generated for the DOM may have an impact on your codebase.
For more information, see the DOM updates for TypeScript 5.2.

[typescript_162] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: `labeledElementDeclarations` May Hold `undefined` Elements

Sumber: ht

`labeledElementDeclarations` May Hold `undefined` Elements
In order to support a mixture of labeled and unlabeled elements, TypeScript's API has changed slightly.
The `labeledElementDeclarations` property of `TupleType` may hold `undefined` for at each position where an element is unlabeled.

[typescript_163] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: `module` and `moduleResolution` Must Match Under Recent Node.js settings

Sumber: ht

`module` and `moduleResolution` Must Match Under Recent Node.js settings
The `--module` and `--moduleResolution` options each support a `node16` and `nodenext` setting.
These are effectively "modern Node.js" settings that should be used on any recent Node.js project.
What we've found is that when these two options don't agree on whether they are using Node.js-related settings, projects are effectively misconfigured.
In TypeScript 5.2, when using `node16` or `nodenext` for either of the `--module` and `--moduleResolution` options, TypeScript now requires the other to have a similar Node.js-related setting.
In cases where the settings diverge, you'll likely get an error message like either
or
So for example `--module esnext --moduleResolution node16` will be rejected -- but you may be better off just using `--module nodenext` alone, or `--module esnext --moduleResolution bundler`.
For more information, see the change here.

Contoh Kode / Perintah CLI:

```
Option 'moduleResolution' must be set to 'NodeNext' (or left unspecified) when option 'module' is set to
'NodeNext'.

// ----------------------------------------

Option 'module' must be set to 'Node16' when option 'moduleResolution' is set to 'Node16'.
```

[typescript_164] TypeScript - Modern Features - TypeScript 5.2

TypeScript 5.2: using Declarations and Explicit Resource Management: Consistent Export Checking for Merged Symbols

Sumber: ht

Consistent Export Checking for Merged Symbols
When two declarations merge, they must agree on whether they are both exported.
Due to a bug, TypeScript missed specific cases in ambient contexts, like in declaration files or `declare module` blocks.
For example, it would not issue an error on a case like the following, where `replaceInFile` is declared once as an

exported function, and one as an un-exported namespace.
In an ambient module, adding an `export { ... }` or a similar construct like `export default ...` implicitly changes whether all declarations are automatically exported.
TypeScript now recognizes these unfortunately confusing semantics more consistently, and issues an error on the fact that all declarations of `replaceInFile` need to agree in their modifiers, and will issue the following error:
For more information, see the change here.

Contoh Kode / Perintah CLI:

```
declare module 'replace-in-file' {
    export function replaceInFile(config: unknown): Promise<unknown[]>;
    export {};

    namespace replaceInFile {
        export function sync(config: unknown): unknown[];
  }
}

// ----------------------------------------

Individual declarations in merged declaration 'replaceInFile' must be all exported or all local.
```

[typescript_165] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Preserved Narrowing in Closures Following Last Assignments

Sumber: ht

Preserved Narrowing in Closures Following Last Assignments
TypeScript can usually figure out a more specific type for a variable based on checks that you might perform.
This process is called narrowing.
One common pain point was that these narrowed types weren't always preserved within function closures.
Here, TypeScript decided that it wasn't "safe" to assume that `url` was *actually* a `URL` object in our callback function because it was mutated elsewhere;
however, in this instance, that arrow function is *always* created after that assignment to `url`, and it's also the *last* assignment to `url`.
TypeScript 5.4 takes advantage of this to make narrowing a little smarter.
When parameters and `let` variables are used in non-hoisted functions, the type-checker will look for a last assignment point.
If one is found, TypeScript can safely narrow from outside the containing function.
What that means is the above example just works now.
Note that narrowing analysis doesn't kick in if the variable is assigned anywhere in a nested function.
This is because there's no way to know for sure whether the function will be called later.
This should make lots of typical JavaScript code easier to express.
You can read more about the change on GitHub.

Contoh Kode / Perintah CLI:

```
function uppercaseStrings(x: string | number) {
    if (typeof x === "string") {
        // TypeScript knows 'x' is a 'string' here.
        return x.toUpperCase();
    }
}

// ----------------------------------------

function getUrls(url: string | URL, names: string[]) {
    if (typeof url === "string") {
        url = new URL(url);
```

```
    }

    return names.map(name => {
        url.searchParams.set("name", name)
        //  ~~~~~~~~~~~~
        // error!
        // Property 'searchParams' does not exist on type 'string | URL'.

        return url.toString();
    });
}

// ----------------------------------------

function printValueLater(value: string | undefined) {
    if (value === undefined) {
        value = "missing!";
    }

    setTimeout(() => {
        // Modifying 'value', even in a way that shouldn't affect
        // its type, will invalidate type refinements in closures.
        value = value;
    }, 500);

    setTimeout(() => {
        console.log(value.toUpperCase());
        //          ~~~~~
        // error! 'value' is possibly 'undefined'.
    }, 1000);
}
```

[typescript_166] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: The `NoInfer` Utility Type

Sumber: ht

The `NoInfer` Utility Type

When calling generic functions, TypeScript is able to infer type arguments from whatever you pass in.

One challenge, however, is that it is not always clear what the "best" type is to infer.

This might lead to TypeScript rejecting valid calls, accepting questionable calls, or just reporting worse error messages when it catches a bug.

For example, let's imagine a `createStreetLight` function that takes a list of color names, along with an optional default color.

What happens when we pass in a `defaultColor` that wasn't in the original `colors` array?

In this function, `colors` is supposed to be the "source of truth" and describe what can be passed to `defaultColor`.

In this call, type inference decided that `"blue"` was just as valid of a type as `"red"` or `"yellow"` or `"green"`.

So instead of rejecting the call, TypeScript infers the type of `C` as `"red" | "yellow" | "green" | "blue"`.

You might say that inference just blue up in our faces!

One way people currently deal with this is to add a separate type parameter that's bounded by the existing type parameter.

This works, but is a little bit awkward because `D` probably won't be used anywhere else in the signature for `createStreetLight`.

While not bad *in this case*, using a type parameter only once in a signature is often a code smell.

That's why TypeScript 5.4 introduces a new `NoInfer` utility type.

Surrounding a type in `NoInfer` gives a signal to TypeScript not to dig in and match against the inner types to find candidates for type inference.

Using `NoInfer`, we can rewrite `createStreetLight` as something like this:
Excluding the type of `defaultColor` from being explored for inference means that `"blue"` never ends up as an inference candidate, and the type-checker can reject it

Contoh Kode / Perintah CLI:

```
function doSomething<T>(arg: T) {
    // ...
}


// We can explicitly say that 'T' should be 'string'.
doSomething<string>("hello!");

// We can also just let the type of 'T' get inferred.
doSomething("hello!");

// ----------------------------------------

function createStreetLight<C extends string>(colors: C[], defaultColor?: C) {
    // ...
}

createStreetLight(["red", "yellow", "green"], "red");

// ----------------------------------------

// Oops! This is undesirable, but is allowed!
createStreetLight(["red", "yellow", "green"], "blue");
```

[typescript_167] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: `Object.groupBy` and `Map.groupBy`

Sumber: ht

`Object.groupBy` and `Map.groupBy`
TypeScript 5.4 adds declarations for JavaScript's new `Object.groupBy` and `Map.groupBy` static methods.
`Object.groupBy` takes an iterable, and a function that decides which "group" each element should be placed in.
The function needs to make a "key" for each distinct group, and `Object.groupBy` uses that key to make an object where every key maps to an array with the original element in it.
So the following JavaScript:
is basically equivalent to writing this:
`Map.groupBy` is similar, but produces a `Map` instead of a plain object.
This might be more desirable if you need the guarantees of `Map`s, you're dealing with APIs that expect `Map`s, or you need to use any kind of key for grouping - not just keys that can be used as property names in JavaScript.
and just as before, you could have created `myObj` in an equivalent way:
Note that in the above example of `Object.groupBy`, the object produced uses all optional properties.
This is because there's no way to guarantee in a general way that *all* the keys were produced by `groupBy`.
Note also that these methods are only accessible by configuring your `target` to `esnext` or adjusting your `lib` settings.
We expect they will eventually be available under a stable `es2024` target.
We'd like to extend a thanks to Kevin Gibbons for adding the declarations to these `groupBy` methods.

Contoh Kode / Perintah CLI:

```
const array = [0, 1, 2, 3, 4, 5];

const myObj = Object.groupBy(array, (num, index) => {
    return num % 2 === 0 ? "even": "odd";
});
```

```
// ----------------------------------------

const myObj = {
    even: [0, 2, 4],
    odd: [1, 3, 5],
};

// ----------------------------------------

const myObj = Map.groupBy(array, (num, index) => {
    return num % 2 === 0 ? "even" : "odd";
});
```

[typescript_168] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Support for `require()` calls in `--moduleResolution bundler` and `--module preserve`

Sumber: ht

Support for `require()` calls in `--moduleResolution bundler` and `--module preserve`

TypeScript has a `moduleResolution` option called `bundler` that is meant to model the way modern bundlers figure out which file an import path refers to.

One of the limitations of the option is that it had to be paired with `--module esnext`, making it impossible to use the `import ... = require(...)` syntax.

That might not seem like a big deal if you're planning on just writing standard ECMAScript `import`s, but there's a difference when using a package with conditional exports.

In TypeScript 5.4, `require()` can now be used when setting the `module` setting to a new option called `preserve`.

Between `--module preserve` and `--moduleResolution bundler`, the two more accurately model what bundlers and runtimes like Bun will allow, and how they'll perform module lookups.

In fact, when using `--module preserve`, the `bundler` option will be implicitly set for `--moduleResolution` (along with `--esModuleInterop` and `--resolveJsonModule`)

Under `--module preserve`, an ECMAScript `import` will always be emitted as-is, and `import ... = require(...)` will be emitted as a `require()` call (though in practice you may not even use TypeScript for emit, since it's likely you'll be using a bundler for your code).

This holds true regardless of the file extension of the containing file.

So the output of this code:

should look something like this:

What this also means is that the syntax you choose directs how conditional exports are matched.

So in the above example, if the `package.json` of `some-package` looks like this:

TypeScript will resolve these paths to `[...]/some-package/esm/foo-from-import.mjs` and `[...]/some-package/cjs/bar-from-require.cjs`.

For more information, you can read up on

Contoh Kode / Perintah CLI:

```
// previously errored
import myModule = require("module/path");

// ----------------------------------------

Under `--module preserve`, an ECMAScript `import` will always be emitted as-is, and `import ... = require(...)` will be emitted as a `require()` call (though in practice you may not even use TypeScript for emit, since it's likely you'll be using a bundler for your code).
This holds true regardless of the file extension of the containing file.
So the output of this code:

// ----------------------------------------

should look something like this:
```

[typescript_169] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Checked Import Attributes and Assertions

Sumber: ht

Checked Import Attributes and Assertions
Import attributes and assertions are now checked against the global `ImportAttributes` type.
This means that runtimes can now more accurately describe the import attributes
This change was provided thanks to Oleksandr Tarasiuk.

Contoh Kode / Perintah CLI:

```
// In some global file.
interface ImportAttributes {
    type: "json";
}

// In some other module
import * as ns from "foo" with { type: "not-json" };
//                             ~~~~~~~~~~
// error!
//
// Type '{ type: "not-json"; }' is not assignable to type 'ImportAttributes'.
//  Types of property 'type' are incompatible.
//    Type '"not-json"' is not assignable to type '"json"'.
```

[typescript_170] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Quick Fix for Adding Missing Parameters

Sumber: ht

Quick Fix for Adding Missing Parameters
TypeScript now has a quick fix to add a new parameter to functions that are called with too many arguments.
!A quick fix being offered when someFunction calls someHelperFunction with 2 more arguments than are expected.
!The missing arguments have been added to someHelperFunction after the quick fix was applied.
This can be useful when threading a new argument through several existing functions, which can be cumbersome today.
This quick fix was provided courtsey of Oleksandr Tarasiuk.

[typescript_171] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Upcoming Changes from TypeScript 5.0 Deprecations

Sumber: ht

Upcoming Changes from TypeScript 5.0 Deprecations
TypeScript 5.0 deprecated the following options and behaviors:
* `charset`
* `target: ES3`
* `importsNotUsedAsValues`
* `noImplicitUseStrict`
* `noStrictGenericChecks`
* `keyofStringsOnly`
* `suppressExcessPropertyErrors`
* `suppressImplicitAnyIndexErrors`
* `out`
* `preserveValueImports`

* `prepend` in project references
* implicitly OS-specific `newLine`

To continue using them, developers using TypeScript 5.0 and other more recent versions have had to specify a new option called `ignoreDeprecations` with the value `"5.0"`.
However, TypScript 5.4 will be the last version in which these will continue to function as normal.
By TypeScript 5.5 (likely June 2024), these will become hard errors, and code using them will need to be migrated away.
For more information, you can read up on this plan on GitHub, which contains suggestions in how to best adapt your codebase.

[typescript_172] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Notable Behavioral Changes

Sumber: ht

Notable Behavioral Changes
This section highlights a set of noteworthy changes that should be acknowledged and understood as part of any upgrade.
Sometimes it will highlight deprecations, removals, and new restrictions.
It can also contain bug fixes that are functionally improvements, but which can also affect an existing build by introducing new errors.

[typescript_173] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: `lib.d.ts` Changes

Sumber: ht

`lib.d.ts` Changes
Types generated for the DOM may have an impact on type-checking your codebase.
For more information, see the DOM updates for TypeScript 5.4.

[typescript_174] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: More Accurate Conditional Type Constraints

Sumber: ht

More Accurate Conditional Type Constraints
The following code no longer allows the second variable declaration in the function `foo`.
Previously, when TypeScript checked the initializer for `second`, it needed to determine whether `IsArray` was assignable to the unit type `false`.
While `IsArray` isn't compatible any obvious way, TypeScript looks at the *constraint* of that type as well.
In a conditional type like `T extends Foo ? TrueBranch : FalseBranch`, where `T` is generic, the type system would look at the constraint of `T`, substitute it in for `T` itself, and decide on either the true or false branch.
But this behavior was inaccurate because it was overly eager.
Even if the constraint of `T` isn't assignable to `Foo`, that doesn't mean that it won't be instantiated with something that is.
And so the more correct behavior is to produce a union type for the constraint of the conditional type in cases where it can't be proven that `T` *never* or *always* extends `Foo.`
TypeScript 5.4 adopts this more accurate behavior.
What this means in practice is that you may begin to find that some conditional type instances are no longer compatible with their branches.
You can read about the specific changes here.

Contoh Kode / Perintah CLI:

```
type IsArray<T> = T extends any[] ? true : false;

function foo<U extends object>(x: IsArray<U>) {
    let first: true = x;    // Error
    let second: false = x;  // Error, but previously wasn't
}
```

[typescript_175] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: More Aggressive Reduction of Intersections Between Type Variables and Primitive Types

Sumber: ht

More Aggressive Reduction of Intersections Between Type Variables and Primitive Types
TypeScript now reduces intersections with type variables and primitives more aggressively, depending on how the type variable's constraint overlaps with those primitives.
For more information, see the change here.

Contoh Kode / Perintah CLI:

```
declare function intersect<T, U>(x: T, y: U): T & U;

function foo<T extends "abc" | "def">(x: T, str: string, num: number) {

    // Was 'T & string', now is just 'T'
    let a = intersect(x, str);

    // Was 'T & number', now is just 'never'
    let b = intersect(x, num)

    // Was '(T & "abc") | (T & "def")', now is just 'T'
    let c = Math.random() < 0.5 ?
        intersect(x, "abc") :
        intersect(x, "def");
}
```

[typescript_176] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Improved Checking Against Template Strings with Interpolations

Sumber: ht

Improved Checking Against Template Strings with Interpolations
TypeScript now more accurately checks whether or not strings are assignable to the placeholder slots of a template string type.
This behavior is more desirable, but may cause breaks in code using constructs like conditional types, where these rule changes are easy to witness.
See this change for more details.

Contoh Kode / Perintah CLI:

```
function a<T extends {id: string}>() {
    let x: `-${keyof T & string}`;

    // Used to error, now doesn't.
    x = "-id";
}
```

[typescript_177] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Errors When Type-Only Imports Conflict with Local Values

Sumber: ht

Errors When Type-Only Imports Conflict with Local Values

Previously, TypeScript would permit the following code under `isolatedModules` if the import to `Something` only referred to a type.

However, it's not safe for single-file compilers to assume whether it's "safe" to drop the `import`, even if the code is guaranteed to fail at runtime.

In TypeScript 5.4, this code will trigger an error like the following:

The fix should be to either make a local rename, or, as the error states, add the `type` modifier to the import:

See more information on the change itself.

Contoh Kode / Perintah CLI:

```
import { Something } from "./some/path";

let Something = 123;

// ----------------------------------------

Import 'Something' conflicts with local value, so must be declared with a type-only import when 'isolatedModules' is enabled.

// ----------------------------------------

import type { Something } from "./some/path";

// or

import { type Something } from "./some/path";
```

[typescript_178] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: New Enum Assignability Restrictions

Sumber: ht

New Enum Assignability Restrictions

When two enums have the same declared names and enum member names, they were previously always considered compatible;

however, when the values were known, TypeScript would silently allow them to have differing values.

TypeScript 5.4 tightens this restriction by requiring the values to be identical when they are known.

Additionally, there are new restrictions for when one of the enum members does not have a statically known value.

In these cases, the other enum must at least be implicitly numeric (e.g. it has no statically resolved initializer), or it is explicitly numeric (meaning TypeScript could resolve the value to something numeric).

Practically speaking, what this means is that string enum members are only ever compatible with other string enums of the same value.

For more information, see the pull request that introduced this change.

Contoh Kode / Perintah CLI:

```
namespace First {
    export enum SomeEnum {
        A = 0,
        B = 1,
    }
}

namespace Second {
```

```
    export enum SomeEnum {
        A = 0,
        B = 2,
    }
}

function foo(x: First.SomeEnum, y: Second.SomeEnum) {
    // Both used to be compatible - no longer the case,
    // TypeScript errors with something like:
    //
    //  Each declaration of 'SomeEnum.B' differs in its value, where '1' was expected but '2' was given.
    x = y;
    y = x;
}

// ----------------------------------------

namespace First {
    export declare enum SomeEnum {
        A,
        B,
    }
}

namespace Second {
    export declare enum SomeEnum {
        A,
        B = "some known string",
    }
}

function foo(x: First.SomeEnum, y: Second.SomeEnum) {
    // Both used to be compatible - no longer the case,
    // TypeScript errors with something like:
    //
    //  One value of 'SomeEnum.B' is the string '"some known string"', and the other is assumed to be an
unknown numeric value.
    x = y;
    y = x;
}
```

[typescript_179] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Name Restrictions on Enum Members

Sumber: ht

Name Restrictions on Enum Members

TypeScript no longer allows enum members to use the names `Infinity`, `-Infinity`, or `NaN`.

See more details here.

Contoh Kode / Perintah CLI:

```
// Errors on all of these:
//
//  An enum member cannot have a numeric name.
enum E {
    Infinity = 0,
    "-Infinity" = 1,
    NaN = 2,
}
```

[typescript_180] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Better Mapped Type Preservation Over Tuples with `any` Rest Elements

Sumber: ht

Better Mapped Type Preservation Over Tuples with `any` Rest Elements
Previously, applying a mapped type with `any` into a tuple would create an `any` element type.
This is undesirable and is now fixed.
For more information, see the fix along with the follow-on discussion around behavioral changes and further tweaks.

Contoh Kode / Perintah CLI:

```
Promise.all(["", ...([] as any)])
    .then((result) => {
        const head = result[0];       // 5.3: any, 5.4: string
        const tail = result.slice(1); // 5.3 any, 5.4: any[]
    });
```

[typescript_181] TypeScript - Modern Features - TypeScript 5.4

TypeScript 5.4: Preserved Narrowing in Closures, NoInfer Utility Type: Emit Changes

Sumber: ht

Emit Changes
While not a breaking change per se, developers may have implicitly taken dependencies on TypeScript's JavaScript or declaration emit outputs.
The following are notable changes.
* Preserve type parameter names more often when shadowed
* Move complex parameter lists of async function into downlevel generator body
* Do not remove binding alias in function declarations
* ImportAttributes should go through the same emit phases when in an ImportTypeNode

[typescript_182] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Inferred Type Predicates

Sumber: ht

Inferred Type Predicates
This section was written by Dan Vanderkam, who implemented this feature in TypeScript 5.5. Thanks Dan!
TypeScript's control flow analysis does a great job of tracking how the type of a variable changes as it moves through your code:
By making you handle the `undefined` case, TypeScript pushes you to write more robust code.
In the past, this sort of type refinement was more difficult to apply to arrays. This would have been an error in all previous versions of TypeScript:
This code is perfectly fine: we've filtered all the `undefined` values out of the list.
But TypeScript hasn't been able to follow along.
With TypeScript 5.5, the type checker is fine with this code:
Note the more precise type for `birds`.
This works because TypeScript now infers a type predicate for the `filter` function.
You can see what's going on more clearly by pulling it out into a standalone function:
`bird is Bird` is the type predicate.
It means that, if the function returns `true`, then it's a `Bird` (if the function returns `false` then it's `undefined`).
The type declarations for `Array.prototype.filter` know about type predicates, so the net result is that you get a more precise type and the code passes the type checker.
TypeScript will infer that a function returns a type predicate if these conditions hold:
1. The function does not have an explicit return type or type predicate annotation.

2. The function has a single `return` statement and no implicit returns.
3. The function does not mutate its parameter.
4. The function returns a `boolean` expression that's tied to a refinement on the parameter.

Generally this works how you'd expect.

Here's a few more examples of inferred type predicates:

Previously, TypeScript would have just inferred that these functions r

Contoh Kode / Perintah CLI:

```
By making you handle the `undefined` case, TypeScript pushes you to write more robust code.

In the past, this sort of type refinement was more difficult to apply to arrays. This would have been an error in all previous versions of TypeScript:

// ----------------------------------------

This code is perfectly fine: we've filtered all the `undefined` values out of the list.
But TypeScript hasn't been able to follow along.

With TypeScript 5.5, the type checker is fine with this code:

// ----------------------------------------

Note the more precise type for `birds`.

This works because TypeScript now infers a [type predicate](https://www.typescriptlang.org/docs/handbook/2/narrowing.html#using-type-predicates) for the `filter` function.
You can see what's going on more clearly by pulling it out into a standalone function:
```

[typescript_183] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Control Flow Narrowing for Constant Indexed Accesses

Sumber: ht

Control Flow Narrowing for Constant Indexed Accesses

TypeScript is now able to narrow expressions of the form `obj[key]` when both `obj` and `key` are effectively constant.

In the above, neither `obj` nor `key` are ever mutated, so TypeScript can narrow the type of `obj[key]` to `string` after the `typeof` check.

For more information, see the implementing pull request here.

Contoh Kode / Perintah CLI:

```
function f1(obj: Record<string, unknown>, key: string) {
    if (typeof obj[key] === "string") {
        // Now okay, previously was error
        obj[key].toUpperCase();
    }
}
```

[typescript_184] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: The JSDoc `@import` Tag

Sumber: ht

The JSDoc `@import` Tag

Today, if you want to import something only for type-checking in a JavaScript file, it is cumbersome.

JavaScript developers can't simply import a type named `SomeType` if it's not there at runtime.

`SomeType` won't exist at runtime, so the import will fail.

Developers can instead use a namespace import instead.
But `./some-module` is still imported at runtime - which might also not be desirable.
To avoid this, developers typically had to use `import(...)` types in JSDoc comments.
If you wanted to reuse the same type in multiple places, you could use a `typedef` to avoid repeating the import.
This helps with local uses of `SomeType`, but it gets repetitive for many imports and can be a bit verbose.
That's why TypeScript now supports a new `@import` comment tag that has the same syntax as ECMAScript imports.
Here, we used named imports.
We could also have written our import as a namespace import.
Because these are just JSDoc comments, they don't affect runtime behavior at all.
We would like to extend a big thanks to Oleksandr Tarasiuk who contributed this change!

Contoh Kode / Perintah CLI:

```
// ./some-module.d.ts
export interface SomeType {
    // ...
}

// ./index.js
import { SomeType } from "./some-module"; // [X] runtime error!

/**
 * @param {SomeType} myValue
 */
function doSomething(myValue) {
    // ...
}

// ----------------------------------------

import * as someModule from "./some-module";

/**
 * @param {someModule.SomeType} myValue
 */
function doSomething(myValue) {
    // ...
}

// ----------------------------------------

/**
 * @param {import("./some-module").SomeType} myValue
 */
function doSomething(myValue) {
    // ...
}
```

[typescript_185] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Regular Expression Syntax Checking

Sumber: ht

Regular Expression Syntax Checking
Until now, TypeScript has typically skipped over most regular expressions in code.
This is because regular expressions technically have an extensible grammar and TypeScript never made any effort to compile regular expressions to earlier versions of JavaScript.
Still, this meant that lots of common problems would go undiscovered in regular expressions, and they would either turn into errors at runtime, or silently fail.

But TypeScript now does basic syntax checking on regular expressions!
This is a simple example, but this checking can catch a lot of common mistakes.
In fact, TypeScript's checking goes slightly beyond syntactic checks.
For instance, TypeScript can now catch issues around backreferences that don't exist.
The same applies to named capturing groups.
TypeScript's checking is now also aware of when certain RegExp features are used when newer than your target version of ECMAScript.
For example, if we use named capturing groups like the above in an ES5 target, we'll get an error.
The same is true for certain regular expression flags as well.
Note that TypeScript's regular expression support is limited to regular expression *literals*.
If you try calling `new RegExp` with a string literal, TypeScript will not check the provided string.
We would like to thank GitHub user graphemecluster who iterated a ton with us to get this feature into TypeScript.

Contoh Kode / Perintah CLI:

```
let myRegex = /@robot(\s+(please|immediately)))? do some task/;
//                                            ~
// error!
// Unexpected ')'. Did you mean to escape it with backslash?

// ----------------------------------------

let myRegex = /@typedef \{import\((.+)\)\.([a-zA-Z_]+)\} \3/u;
//                                                      ~
// error!
// This backreference refers to a group that does not exist.
// There are only 2 capturing groups in this regular expression.

// ----------------------------------------

let myRegex = /@typedef \{import\((?<importPath>.+)\)\.(?<importedEntity>[a-zA-Z_]+)\} \k<namedImport>/;
//                                                                                      ~~~~~~~~~~~
// error!
// There is no capturing group named 'namedImport' in this regular expression.
```

[typescript_186] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Support for New ECMAScript `Set` Methods

Sumber: ht

Support for New ECMAScript `Set` Methods
TypeScript 5.5 declares new proposed methods for the ECMAScript `Set` type.
Some of these methods, like `union`, `intersection`, `difference`, and `symmetricDifference`, take another `Set` and return a new `Set` as the result.
The other methods, `isSubsetOf`, `isSupersetOf`, and `isDisjointFrom`, take another `Set` and return a `boolean`.
None of these methods mutate the original `Set`s.
Here's a quick example of how you might use these methods and how they behave:
We'd like to thank Kevin Gibbons who not only co-championed the feature in ECMAScript, but also provided the declarations for `Set`, `ReadonlySet`, and `ReadonlySetLike` in TypeScript!

Contoh Kode / Perintah CLI:

```
let fruits = new Set(["apples", "bananas", "pears", "oranges"]);
let applesAndBananas = new Set(["apples", "bananas"]);
let applesAndOranges = new Set(["apples", "oranges"]);
let oranges = new Set(["oranges"]);
let emptySet = new Set();

////
// union
```

```
////

// Set(4) {'apples', 'bananas', 'pears', 'oranges'}
console.log(fruits.union(oranges));

// Set(3) {'apples', 'bananas', 'oranges'}
console.log(applesAndBananas.union(oranges));

////
// intersection
////

// Set(2) {'apples', 'bananas'}
console.log(fruits.intersection(applesAndBananas));

// Set(0) {}
console.log(applesAndBananas.intersection(oranges));

// Set(1) {'apples'}
console.log(applesAndBananas.intersection(applesAndOranges));

////
// difference
////

// Set(3) {'apples', 'bananas', 'pears'}
console.log(fruits.difference(oranges));

// Set(2) {'pears', 'oranges'}
console.log(fruits.difference(applesAndBananas));

// Set(1) {'bananas'}
console.log(applesAndBananas.difference(applesAndOranges));

////
// symmetricDifference
////

// Set(2) {'bananas', 'oranges'}
console.log(applesAndBananas.symmetricDifference(applesAndOranges)); // no apples

////
// isDisjointFrom
////

// true
console.log(applesAndBananas.isDisjointFrom(oranges));

// false
console.log(applesAndBananas.isDisjointFrom(applesAndOranges));

// true
console.log(fruits.isDisjointFrom(emptySet));

// true
console.log(emptySet.isDisjointFrom(emptySet));

////
// isSubsetOf
////

// true
```

```
console.log(applesAndBananas.isSubsetOf(fruits));

// false
console.log(fruits.isSubsetOf(applesAndBananas));

// false
console.log(applesAndBananas.isSubsetOf(oranges));

// true
console.log(fruits.isSubsetOf(fruits));

// true
console.log(emptySet.isSubsetOf(fruits));

////
// isSupersetOf
////

// true
console.log(fruits.isSupersetOf(applesAndBananas));

// false
console.log(applesAndBananas.isSupersetOf(fruits));

// false
console.log(applesAndBananas.isSupersetOf(oranges));

// true
console.log(fruits.isSupersetOf(fruits));

// false
console.log(emptySet.isSupersetOf(fruits));
```

[typescript_187] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Isolated Declarations

Sumber: ht

Isolated Declarations

This section was co-authored by Rob Palmer who supported the design of isolated declarations.

Declaration files (a.k.a. `.d.ts` files) describe the shape of existing libraries and modules to TypeScript.

This lightweight description includes the library's type signatures and excludes implementation details such as the function bodies.

They are published so that TypeScript can efficiently check your usage of a library without needing to analyse the library itself.

Whilst it is possible to handwrite declaration files, if you are authoring typed code, it's much safer and simpler to let TypeScript generate them automatically from source files using `--declaration`.

The TypeScript compiler and its APIs have always had the job of generating declaration files;

however, there are some use-cases where you might want to use other tools, or where the traditional build process doesn't scale.

[typescript_188] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Use-case: Faster Declaration Emit Tools

Sumber: ht

Use-case: Faster Declaration Emit Tools

Imagine if you wanted to create a faster tool to generate declaration files, perhaps as part of a publishing service or a new bundler.

Whilst there is a thriving ecosystem of blazing fast tools that can turn TypeScript into JavaScript, the same is not true for turning TypeScript into declaration files.

The reason is that TypeScript's inference allows us to write code without explicitly declaring types, meaning declaration emit can be complex.

Let's consider a simple example of a function that adds two imported variables.

Even if the only thing we want to do is generate `add.d.ts`, TypeScript needs to crawl into another imported file (`util.ts`), infer that the type of `one` and `two` are strings, and then calculate that the `+` operator on two strings will lead to a `string` return type.

While this inference is important for the developer experience, it means that tools that want to generate declaration files would need to replicate parts of the type-checker including inference and the ability to resolve module specifiers to follow the imports.

Contoh Kode / Perintah CLI:

```
// util.ts
export let one = "1";
export let two = "2";

// add.ts
import { one, two } from "./util";
export function add() { return one + two; }

// ---------------------------------------

// add.d.ts
export declare function add(): string;
```

[typescript_189] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Use-case: Parallel Declaration Emit and Parallel Checking

Sumber: ht

Use-case: Parallel Declaration Emit and Parallel Checking

Imagine if you had a monorepo containing many projects and a multi-core CPU that just wished it could help you check your code faster.

Wouldn't it be great if we could check all those projects at the same time by running each project on a different core?

Unfortunately we don't have the freedom to do all the work in parallel.

The reason is that we have to build those projects in dependency order, because each project is checking against the declaration files of their dependencies.

So we must build the dependency first to generate the declaration files.

TypeScript's project references feature works the same way, building the set of projects in "topological" dependency order.

As an example, if we have two projects called `backend` and `frontend`, and they both depend on a project called `core`, TypeScript can't start type-checking either `frontend` or `backend` until `core` has been built and its declaration files have been generated.

!frontend and backend point to core, other stuff might point to each of those

In the above graph, you can see that we have a bottleneck.

Whilst we can build `frontend` and `backend` in parallel, we need to first wait for `core` to finish building before either

can start.
How could we improve upon this?
Well, if a fast tool could generate all those declaration files for `core` *in parallel*, TypeScript then could immediately follow that by type-checking `core`, `frontend`, and `backend` also *in parallel*.

[typescript_190] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Solution: Explicit Types!

Sumber: ht

Solution: Explicit Types!
The common requirement in both use-cases is that we need a cross-file type-checker to generate declaration files.
Which is a lot to ask from the tooling community.
As a more complex example, if we want a declaration file for the following code...
...we would need to generate a signature for `foo`.
Well that requires looking at the implementation of `foo`.
`foo` just returns `x`, so getting the type of `x` requires looking at the implementation of `add`.
But that might require looking at the implementation of `add`'s dependencies, and so on.
What we're seeing here is that generating declaration files requires a whole lot of logic to figure out the types of different places that might not even be local to the current file.
Still, for developers looking for fast iteration time and fully parallel builds, there is another way of thinking about this problem.
A declaration file only requires the types of the public API of a module - in other words, the types of the things that are exported.
If, controversially, developers are willing to explicitly write out the types of the things they export, tools could generate declaration files without needing to look at the implementation of the module - and without reimplementing a full type-checker.
This is where the new `--isolatedDeclarations` option comes in.
`--isolatedDeclarations` reports errors when a module can't be reliably transformed without a type-checker.
More plainly, it makes TypeScript report errors if you have a file that isn't sufficiently annotated on its exports.
That means in the above example, we would see an error like the following:

Contoh Kode / Perintah CLI:

```
import { add } from "./add";

const x = add();

export function foo() {
    return x;
}

// ----------------------------------------

export function foo() {
//              ~~~
// error! Function must have an explicit
// return type annotation with --isolatedDeclarations.
    return x;
}
```

[typescript_191] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Why are errors desirable?

Sumber: ht

Why are errors desirable?
Because it means that TypeScript can
1. Tell us up-front whether other tools will have issues with generating declaration files
2. Provide a quick fix to help add these missing annotations.
This mode doesn't require annotations *everywhere* though.
For locals, these can be ignored, since they don't affect the public API.
For example, the following code would **not** produce an error:
There are also certain expressions where the type is "trivial" to calculate.

Contoh Kode / Perintah CLI:

```
import { add } from "./add";

const x = add("1", "2"); // no error on 'x', it's not exported.

export function foo(): string {
    return x;
}

// ----------------------------------------

// No error on 'x'.
// It's trivial to calculate the type is 'number'
export let x = 10;

// No error on 'y'.
// We can get the type from the return expression.
export function y() {
    return 20;
}

// No error on 'z'.
// The type assertion makes it clear what the type is.
export function z() {
    return Math.max(x, y()) as number;
}
```

[typescript_192] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Using `isolatedDeclarations`

Sumber: ht

Using `isolatedDeclarations`
`isolatedDeclarations` requires that either the `declaration` or `composite` flags are also set.
Note that `isolatedDeclarations` does not change how TypeScript performs emit - just how it reports errors.
Importantly, and similar to `isolatedModules`, enabling the feature in TypeScript won't immediately bring about the potential benefits discussed here.
So please be patient and look forward to future developments in this space.
Keeping tool authors in mind, we should also recognize that today, not all of TypeScript's declaration emit can be easily replicated by other tools wanting to use it as a guide.
That's something we're actively working on improving.
On top of this, isolated declarations are still a new feature, and we're actively working on improving the experience.
Some scenarios, like using computed property declarations in classes and object literals, are not *yet* supported under

`isolatedDeclarations`.
Keep an eye on this space, and feel free to provide us with feedback.
We also feel it is worth calling out that `isolatedDeclarations` should be adopted on a case-by-case basis.
There are some developer ergonomics that are lost when using `isolatedDeclarations`, and thus it may not be the right choice if your setup is not leveraging the two scenarios mentioned earlier.
For others, the work on `isolatedDeclarations` has already uncovered many optimizations and opportunities to unlock different parallel build strategies.
In the meantime, if you're willing to make the trade-offs, we believe `isolatedDeclarations` can be a powerful tool to speed up your build process as external tooling becomes more widely available.
For more information, read up on the Isolated Declarations: State of the Feature discussion on the TypeScript issue t

[typescript_193] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Credit

Sumber: ht

Credit
Work on `isolatedDeclarations` has been a long-time collaborative effort between the TypeScript team and the infrastructure and tooling teams within Bloomberg and Google.
Individuals like Hana Joo from Google who implemented the quick fix for isolated declaration errors (more on that soon), as well as Ashley Claymore, Jan Kühle, Lisa Velden, Rob Palmer, and Thomas Chetwin have been involved in discussion, specification, and implementation for many months.
But we feel it is specifically worth calling out the tremendous amount of work provided by Titian Cernicova-Dragomir from Bloomberg.
Titian has been instrumental in driving the implementation of `isolatedDeclarations` and has been a contributor to the TypeScript project for years prior.
While the feature involved many changes, you can see the core work for Isolated Declarations here.

[typescript_194] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: The `${configDir}` Template Variable for Configuration Files

Sumber: ht

The `${configDir}` Template Variable for Configuration Files
It's common in many codebases to reuse a shared `tsconfig.json` file that acts as a "base" for other configuration files.
This is done by using the `extends` field in a `tsconfig.json` file.
One of the issues with this is that all paths in the `tsconfig.json` file are relative to the location of the file itself.
This means that if you have a shared `tsconfig.base.json` file that is used by multiple projects, relative paths often won't be useful in the derived projects.
For example, imagine the following `tsconfig.base.json`:
If author's intent was that every `tsconfig.json` that extends this file should
1. output to a `dist` directory relative to the derived `tsconfig.json` , and
1. have a `custom-types` directory relative to the derived `tsconfig.json`,
then this would not work.
The `typeRoots` paths would be relative to the location of the shared `tsconfig.base.json` file, not the project that extends it.
Each project that extends this shared file would need to declare its own `outDir` and `typeRoots` with identical contents.
This could be frustrating and hard to keep in sync between projects, and while the example above is using `typeRoots`, this is a common problem for `paths` and other options.
To solve this, TypeScript 5.5 introduces a new template variable `${configDir}`.
When `${configDir}` is written in certain path fields of a `tsconfig.json` or `jsconfig.json` files, this variable is substituted with the containing directory of the configuration file in a given compilation.
This means that the above `tsconfig.base.json` could be rewritten as:

Now, when a project extends this file, the paths will be relative to the derived `tsconfig.json`, not the shared `tsconfig.base.json` file.
This makes it ea

Contoh Kode / Perintah CLI:

```
{
    "extends": "../../tsconfig.base.json",
    "compilerOptions": {
        "outDir": "./dist"
    }
}

// ----------------------------------------

If author's intent was that every `tsconfig.json` that extends this file should

1. output to a `dist` directory relative to the derived `tsconfig.json` , and
1. have a `custom-types` directory relative to the derived `tsconfig.json`,

then this would not work.
The `typeRoots` paths would be relative to the location of the shared `tsconfig.base.json` file, not the project that extends it.
Each project that extends this shared file would need to declare its own `outDir` and `typeRoots` with identical contents.
This could be frustrating and hard to keep in sync between projects, and while the example above is using `typeRoots`, this is a common problem for `paths` and other options.

To solve this, TypeScript 5.5 introduces a new template variable `${configDir}`.
When `${configDir}` is written in certain path fields of a `tsconfig.json` or `jsconfig.json` files, this variable is substituted with the containing directory of the configuration file in a given compilation.
This means that the above `tsconfig.base.json` could be rewritten as:
```

[typescript_195] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Consulting `package.json` Dependencies for Declaration File Generation

Sumber: ht

Consulting `package.json` Dependencies for Declaration File Generation
Previously, TypeScript would often issue an error message like
This was often due to TypeScript's declaration file generation finding itself in the contents of files that were never explicitly imported in a program.
Generating an import to such a file could be risky if the path ended up being relative.
Still, for codebases with explicit dependencies in the `dependencies` (or `peerDependencies` and `optionalDependencies`) of a `package.json`, generating such an import should be safe under certain resolution modes.
So in TypeScript 5.5, we're more lenient when that's the case, and many occurrences of this error should disappear.
See this pull request for more details on the change.

Contoh Kode / Perintah CLI:

```
The inferred type of "X" cannot be named without a reference to "Y". This is likely not portable. A type annotation is necessary.
```

[typescript_196] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Editor and Watch-Mode Reliability Improvements

Sumber: ht

Editor and Watch-Mode Reliability Improvements
TypeScript has either added some new functionality or fixed existing logic that makes `--watch` mode and TypeScript's editor integration feel more reliable.
That should hopefully translate to fewer TSServer/editor restarts.

[typescript_197] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Correctly Refresh Editor Errors in Configuration Files

Sumber: ht

Correctly Refresh Editor Errors in Configuration Files
TypeScript can generate errors for `tsconfig.json` files;
however, those errors are actually generated from loading a project, and editors typically don't directly request those errors for `tsconfig.json` files.
While this sounds like a technical detail, it means that when all errors issued in a `tsconfig.json` are fixed, TypeScript doesn't issue a new fresh empty set of errors, and users are left with stale errors unless they reload their editor.
TypeScript 5.5 now intentionally issues an event to clear these out.
See more here.

[typescript_198] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Better Handling for Deletes Followed by Immediate Writes

Sumber: ht

Better Handling for Deletes Followed by Immediate Writes
Instead of overwriting files, some tools will opt to delete them and then create new files from scratch.
This is the case when running `npm ci`, for instance.
While this can be efficient for those tools, it can be problematic for TypeScript's editor scenarios where deleting a watched might dispose of it and all of its transitive dependencies.
Deleting and creating a file in quick succession could lead to TypeScript tearing down an entire project and then rebuilding it from scratch.
TypeScript 5.5 now has a more nuanced approach by keeping parts of a deleted project around until it picks up on a new creation event.
This should make operations like `npm ci` work a lot better with TypeScript.
See more information on the approach here.

[typescript_199] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Symlinks are Tracked in Failed Resolutions

Sumber: ht

Symlinks are Tracked in Failed Resolutions
When TypeScript fails to resolve a module, it will still need to watch for any failed lookup paths in case the module is added later.
Previously this was not done for symlinked directories, which could cause reliability issues in monorepo-like scenarios when a build occurred in one project but was not witnessed in the other.
This should be fixed in TypeScript 5.5, and means you won't need to restart your editor as often.
See more information here.

[typescript_200] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Project References Contribute to Auto-Imports

Sumber: ht

Project References Contribute to Auto-Imports
Auto-imports no longer requires at least one explicit import to dependent projects in a project reference setup.
Instead, auto-import completions should just work across anything you've listed in the `references` field of your `tsconfig.json`.
See more on the implementing pull request.

[typescript_201] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Performance and Size Optimizations

Sumber: ht

Performance and Size Optimizations

[typescript_202] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Monomorphized Objects in Language Service and Public API

Sumber: ht

Monomorphized Objects in Language Service and Public API
In TypeScript 5.0, we ensured that our `Node` and `Symbol` objects had a consistent set of properties with a consistent initialization order.
Doing so helps reduce polymorphism in different operations, which allows runtimes to fetch properties more quickly.
By making this change, we witnessed impressive speed wins in the compiler;
however, most of these changes were performed on internal allocators for our data structures.
The language service, along with TypeScript's public API, uses a different set of allocators for certain objects.
This allowed the TypeScript compiler to be a bit leaner, as data used only for the language service would never be used in the compiler.
In TypeScript 5.5, the same monomorphization work has been done for the language service and public API.
What this means is that your editor experience, and any build tools that use the TypeScript API, will get a decent amount faster.
In fact, in our benchmarks, we've seen a **5-8% speedup in build times** when using the public TypeScript API's allocators, and **language service operations getting 10-20% faster**.
While this does imply an increase in memory, we believe that tradeoff is worth it and hope to find ways to reduce that memory overhead.
Things should feel a lot snappier now.
For more information, see the change here.

[typescript_203] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Monomorphized Control Flow Nodes

Sumber: ht

Monomorphized Control Flow Nodes
In TypeScript 5.5, nodes of the control flow graph have been monomorphized so that they always hold a consistent shape.
By doing so, check times will often be reduced by about 1%.
See this change here.

[typescript_204] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Optimizations on our Control Flow Graph

Sumber: ht

Optimizations on our Control Flow Graph

In many cases, control flow analysis will traverse nodes that don't provide any new information.
We observed that in the absence of any early termination or effects in the antecedents (or "dominators") of certain nodes meant that those nodes could always be skipped over.
As such, TypeScript now constructs its control flow graphs to take advantage of this by linking to an earlier node that *does* provide interesting information for control flow analysis.
This yields a flatter control flow graph, which can be more efficient to traverse.
This optimization has yielded modest gains, but with up to 2% reductions in build time on certain codebases.
You can read more here.

[typescript_205] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Skipped Checking in `transpileModule` and `transpileDeclaration`

Sumber: ht

Skipped Checking in `transpileModule` and `transpileDeclaration`

TypeScript's `transpileModule` API can be used for compiling a single TypeScript file's contents into JavaScript.
Similarly, the `transpileDeclaration` API (see below) can be used to generate a declaration file for a single TypeScript file.
One of the issues with these APIs is that TypeScript internally would perform a full type-checking pass over the entire contents of the file before emitting the output.
This was necessary to collect certain information which would later be used for the emit phase.
In TypeScript 5.5, we've found a way to avoid performing a full check, only lazily collecting this information as necessary, and `transpileModule` and `transpileDeclaration` both enable this functionality by default.
As a result, tools that integrate with these APIs, like ts-loader with `transpileOnly` and ts-jest, should see a noticeable speedup.
In our testing, we generally witness around a 2x speed-up in build time using `transpileModule`.

[typescript_206] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: TypeScript Package Size Reduction

Sumber: ht

TypeScript Package Size Reduction

Further leveraging our transition to modules in 5.0, we've significantly reduced TypeScript's overall package size by making `tsserver.js` and `typingsInstaller.js` import from a common API library instead of having each of them produce standalone bundles.
This reduces TypeScript's size on disk from 30.2 MB to 20.4 MB, and reduces its packed size from 5.5 MB to 3.7 MB!

[typescript_207] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Node Reuse in Declaration Emit

Sumber: ht

Node Reuse in Declaration Emit

As part of the work to enable `isolatedDeclarations`, we've substantially improved how often TypeScript can directly copy your input source code when producing declaration files.
For example, let's say you wrote

Note that the union types are equivalent, but the order of the union is different.
When emitting the declaration file, TypeScript has two equivalent output possibilities.
The first is to use a consistent canonical representation for each type:
The second is to re-use the type annotations exactly as written:
The second approach is generally preferable for a few reasons:
* Many equivalent representations still encode some level of intent that is better to preserve in the declaration file
* Producing a fresh representation of a type can be somewhat expensive, so avoiding is better
* User-written types are usually shorter than generated type representations
In 5.5, we've greatly improved the number of places where TypeScript can correctly identify places where it's safe and correct to print back types exactly as they were written in the input file.
Many of these cases are invisible performance improvements - TypeScript would generate fresh sets of syntax nodes and serialize them into a string.
Instead, TypeScript can now operate over the original syntax nodes directly, which is much cheaper and faster.

Contoh Kode / Perintah CLI:

```
export const strBool: string | boolean = "hello";
export const boolStr: boolean | string = "world";

// ---------------------------------------

export const strBool: string | boolean;
export const boolStr: string | boolean;

// ---------------------------------------

export const strBool: string | boolean;
export const boolStr: boolean | string;
```

[typescript_208] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Caching Contextual Types from Discriminated Unions

Sumber: ht

Caching Contextual Types from Discriminated Unions
When TypeScript asks for the contextual type of an expression like an object literal, it will often encounter a union type.
In those cases, TypeScript tries to filter out members of the union based on known properties with well known values (i.e. discriminant properties).
This work can be fairly expensive, especially if you end up with an object consisting of many many properties.
In TypeScript 5.5, much of the computation is cached once so that TypeScript doesn't need to recompute it for every property in the object literal.
Performing this optimization shaved 250ms off of compiling the TypeScript compiler itself.

[typescript_209] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Easier API Consumption from ECMAScript Modules

Sumber: ht

Easier API Consumption from ECMAScript Modules
Previously, if you were writing an ECMAScript module in Node.js, named imports were not available from the `typescript` package.
This is because cjs-module-lexer did not recognize the pattern of TypeScript's generated CommonJS code.
This has been fixed, and users can now use named imports from the TypeScript npm package with ECMAScript modules in Node.js.
For more information, see the change here.

Contoh Kode / Perintah CLI:

```
import { createSourceFile } from "typescript"; // [X] error

import * as ts from "typescript";
ts.createSourceFile // [X] undefined???

ts.default.createSourceFile // ? works - but ugh!

// ----------------------------------------

import { createSourceFile } from "typescript"; // ? works now!

import * as ts from "typescript";
ts.createSourceFile // ? works now!
```

[typescript_210] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: The `transpileDeclaration` API

Sumber: ht

The `transpileDeclaration` API
TypeScript's API exposes a function called `transpileModule`.
It's intended to make it easy to compile a single file of TypeScript code.
Because it doesn't have access to an entire *program*, the caveat is that it may not produce the right output if the code violates any errors under the `isolatedModules` option.
In TypeScript 5.5, we've added a new similar API called `transpileDeclaration`.
This API is similar to `transpileModule`, but it's specifically designed to generate a single *declaration file* based on some input source text.
Just like `transpileModule`, it doesn't have access to a full program, and a similar caveat applies: it only generates an accurate declaration file if the input code is free of errors under the new `isolatedDeclarations` option.
If desired, this function can be used to parallelize declaration emit across all files under `isolatedDeclarations` mode.
For more information, see the implementation here.

[typescript_211] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Notable Behavioral Changes

Sumber: ht

Notable Behavioral Changes
This section highlights a set of noteworthy changes that should be acknowledged and understood as part of any upgrade.
Sometimes it will highlight deprecations, removals, and new restrictions.
It can also contain bug fixes that are functionally improvements, but which can also affect an existing build by introducing new errors.

[typescript_212] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Disabling Features Deprecated in TypeScript 5.0

Sumber: ht

Disabling Features Deprecated in TypeScript 5.0
TypeScript 5.0 deprecated the following options and behaviors:
* `charset`
* `target: ES3`

* `importsNotUsedAsValues`
* `noImplicitUseStrict`
* `noStrictGenericChecks`
* `keyofStringsOnly`
* `suppressExcessPropertyErrors`
* `suppressImplicitAnyIndexErrors`
* `out`
* `preserveValueImports`
* `prepend` in project references
* implicitly OS-specific `newLine`

To continue using the deprecated options above, developers using TypeScript 5.0 and other more recent versions have had to specify a new option called `ignoreDeprecations` with the value `"5.0"`.
In TypeScript 5.5, these options no longer have any effect.
To help with a smooth upgrade path, you may still specify them in your tsconfig, but these will be an error to specify in TypeScript 6.0.
See also the Flag Deprecation Plan which outlines our deprecation strategy.
More information around these deprecation plans is available on GitHub, which contains suggestions in how to best adapt your codebase.

[typescript_213] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: `lib.d.ts` Changes

Sumber: ht

`lib.d.ts` Changes
Types generated for the DOM may have an impact on type-checking your codebase.
For more information, see the DOM updates for TypeScript 5.5.

[typescript_214] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Stricter Parsing for Decorators

Sumber: ht

Stricter Parsing for Decorators
Since TypeScript originally introduced support for decorators, the specified grammar for the proposal has been tightened up.
TypeScript is now stricter about what forms it allows.
While rare, existing decorators may need to be parenthesized to avoid errors.
See more information on the change here.

Contoh Kode / Perintah CLI:

```
class DecoratorProvider {
    decorate(...args: any[]) { }
}

class D extends DecoratorProvider {
    m() {
        class C {
            @super.decorate // [X] error
            method1() { }

            @(super.decorate) // ? okay
            method2() { }
        }
    }
```

```
}
```

[typescript_215] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: `undefined` is No Longer a Definable Type Name

Sumber: ht

`undefined` is No Longer a Definable Type Name
TypeScript has always disallowed type alias names that conflict with built-in types:
Due to a bug, this logic didn't also apply to the built-in type `undefined`.
In 5.5, this is now correctly identified as an error:
Bare references to type aliases named `undefined` never actually worked in the first place.
You could define them, but you couldn't use them as an unqualified type name.
For more information, see the change here.

Contoh Kode / Perintah CLI:

```
// Illegal
type null = any;
// Illegal
type number = any;
// Illegal
type object = any;
// Illegal
type any = any;

// ----------------------------------------

// Now also illegal
type undefined = any;

// ----------------------------------------

export type undefined = string;
export const m: undefined = "";
//           ^
// Errors in 5.4 and earlier - the local definition of 'undefined' was not even consulted.
```

[typescript_216] TypeScript - Modern Features - TypeScript 5.5

TypeScript 5.5: Inferred Type Predicates, Regular Expression Syntax Checking: Simplified Reference Directive Declaration Emit

Sumber: ht

Simplified Reference Directive Declaration Emit
When producing a declaration file, TypeScript would synthesize a reference directive when it believed one was required.
For example, all Node.js modules are declared ambiently, so cannot be loaded by module resolution alone.
A file like:
Would emit a declaration file like:
Even though the reference directive never appeared in the original source.
Similarly, TypeScript also *removed* reference directives that it did not believe needed to be a part of the output.
For example, let's imagine we had a reference directive to `jest`;
however, imagine the reference directive isn't necessary to generate the declaration file.
TypeScript would simply drop it.
So in the following example:
TypeScript would still emit:
In the course of working on `isolatedDeclarations`, we realized that this logic was untenable for anyone attempting to implement a declaration emitter without type checking or using more than a single file's context.

This behavior is also hard to understand from a user's perspective; whether or not a reference directive appeared in the emitted file seems inconsistent and difficult to predict unless you understand exactly what's going on during typechecking.
To prevent declaration emit from being different when `isolatedDeclarations` was enabled, we knew that our emit needed to change.
Through experimentation, we found that nearly all cases where TypeScript synthesized reference directives were just to pull in `node` or `react`.
These are cases where the expectation is that a downstream user already references those types through tsconfig.json `"types"` or library imports, so no longer synthesizing these reference directives would be unlikely to break anyone.
It's worth noting that this is already how it works for `lib

Contoh Kode / Perintah CLI:

```
Would emit a declaration file like:

// ----------------------------------------

Even though the reference directive never appeared in the original source.

Similarly, TypeScript also *removed* reference directives that it did not believe needed to be a part of
the output.
For example, let's imagine we had a reference directive to `jest`;
however, imagine the reference directive isn't necessary to generate the declaration file.
TypeScript would simply drop it.
So in the following example:

// ----------------------------------------

TypeScript would still emit:
```